



Wzorce projektowe

Jerzy Grynczewski

Plan bloku



Plan bloku

- ◊ Wstęp
- ◊ Paradygmat obiektowe
 - Podstawy
 - Relacje
 - Interfejsy
- ◊ Zasady projektowania
 - OOA, OOD, OOP
 - UML
 - YAGNI, KISS, DRY
 - GRASP
 - SOLID
 - STUPID i CUPID
- ◊ Wzorce projektowe
 - GoF
- ◊ Metryki kodu
- ◊ "Zapachy" kodu
- ◊ PEP8 i dobre praktyki

W_{step}

A thin, vertical blue line is positioned to the right of the text W_{step} .

Co to jest wzorzec
projektowy

Wzorzec projektowy to modelowe rozwiązanie
powszechnie występującego problemu projektowego.
Opisuje problem i ogólne podejście do jego rozwiązania.

Co to jest wzorzec projektowy

"Each pattern describes a problem which occurs over and over again in our environment and then describes the core of the solution to the problem".

Christopher Alexander (architect and design theorist not a software developer)

A Pattern Language, Oxford University Press, New York, 1977.

Przykłady wzorców projektowych



Budynki

Przykłady wzorców projektowych



Budynki



Kody elektroniczne
i hydrauliczne

Przykłady wzorców projektowych



Budynki



Kody elektroniczne
i hydrauliczne



Samochody

Przykłady wzorców projektowych



Budynki



Kody elektroniczne i
hydrauliczne



Samochody



Interfejsy telefonów
komórkowych

Co dają nam
wzorce projektowe

Wzorce projektowe dają nam pewność, że wyniki
naszej pracy są spójne, niezawodne i zrozumiałe.

Paradygmat obiektowy

Paradygmat obiektowy

- Klasy
- Obiekty
- Atrybuty (obektowe, klasowe)
- Metody (obektowe, klasowe, statyczne)

Podstawowe cechy paradygmatu
obiektowego

- Enkapsulacja (aka hermentyzacja, kapsułkowanie)
- Abstrakcja
- Dziedziczenie
- Polimorfizm

Relacje (aka powiązania)

- Interfejs (IMPLEMENTED-A)
- Dziedziczenie (IS-A)
- Kompozycja (HAS-A)
- Zależność (USE-A)

Interfejsy

Interfejsy w Pythonie

I w SOLID

Interfejsy w Pythonie

I w SOLID

Wsparcie w Java,
C#, Visual Basic
dla definicji
interfejsu

Interfejsy w Pythonie

I w SOLID

Wsparcie w Java,
C#, Visual Basic
dla definicji
interfejsu

Wsparcie w C++ za
pomocą klas
abstrakcyjnych

Interfejsy w Pythonie

I w SOLID

Wsparcie w Java,
C#, Visual Basic
dla definicji
interfejsu

Wsparcie w C++ za
pomocą klas
abstrakcyjnych

Domyślnie brak
wsparcia w
Pythonie

Interfejsy w Pythonie

I w SOLID

Wsparcie w Java,
C#, Visual Basic
dla definicji
interfejsu

Wsparcie w C++ za
pomocą klas
abstrakcyjnych

Domyślnie brak
wsparcia w
Pythonie

Wprowadzone
przez PEP 3119
za pomocą klas
abstrakcyjnych

Interfejsy w Pythonie

I w SOLID

Wsparcie w Java,
C#, Visual Basic
dla definicji
interfejsu

Wsparcie w C++ za
pomocą klas
abstrakcyjnych

Domyślnie brak
wsparcia w
Pythonie

Wprowadzone
przez PEP 3119
za pomocą klas
abstrakcyjnych

Pierwszy raz
pojawiły się w
Pythonie 2.6 i 3

Definiowanie abstrakcyjnych klas bazowych

```
import abc

class MyABC(abc.ABC):
    """Abstract Base Class Definition"""

    @abc.abstractmethod
    def do_something(self, value):
        """Required method"""

    @property
    @abc.abstractmethod
    def some_property(self):
        """Required property"""
```

Definiowanie abstrakcyjnych klas bazowych



Moduł abc

```
import abc
```

```
class MyABC(abc.ABC):  
    """Abstract Base Class Definition"""
```

```
    @abc.abstractmethod  
    def do_something(self, value):  
        """Required method"""
```

```
    @property  
    @abc.abstractmethod  
    def some_property(self):  
        """Required property"""
```


Definiowanie abstrakcyjnych klas bazowych

Moduł abc

```
import abc
```

Klasa abstrakcyjna

```
class MyABC(abc.ABC):  
    """Abstract Base Class Definition"""
```

```
    @abc.abstractmethod  
    def do_something(self, value):  
        """Required method"""
```

```
    @property  
    @abc.abstractmethod  
    def some_property(self):  
        """Required property"""
```

Definiowanie abstrakcyjnych klas bazowych

Moduł abc

```
import abc
```

Klasa abstrakcyjna

```
class MyABC(abc.ABC):  
    """Abstract Base Class Definition"""
```

Metoda abstrakcyjna

```
@abc.abstractmethod  
def do_something(self, value):  
    """Required method"""
```

```
@property  
@abc.abstractmethod  
def some_property(self):  
    """Required property"""
```


Definiowanie abstrakcyjnych klas bazowych

Moduł abc

```
import abc
```

Klasa abstrakcyjna

```
class MyABC(abc.ABC):  
    """Abstract Base Class Definition"""
```

Metoda abstrakcyjna

```
@abc.abstractmethod  
def do_something(self, value):  
    """Required method"""
```

Property abstrakcyjne

```
@property  
@abc.abstractmethod  
def some_property(self):  
    """Required property"""
```

Implementacja konkretnej klasy

Dziedziczy z ABC

```
class MyClass(MyABC):  
    """Implementation of MyABC"""  
  
    def __init__(self, value=None):  
        self._my_prop = value  
  
    def do_something(self, value):  
        """Implementation of abstract method"""  
        self._myprop *= 2+value  
  
    @property  
    def some_property(self):  
        """Implementation of abstract property"""  
        return self._myprop
```

Implementacja konkretnej klasy

Dziedziczy z ABC

```
class MyClass(MyABC):  
    """Implementation of MyABC"""
```

Standardowy konstruktor

```
def __init__(self, value=None):  
    self._my_prop = value
```

```
def do_something(self, value):  
    """Implementation of abstract method"""  
    self._myprop *= 2+value
```

```
@property  
def some_property(self):  
    """Implementation of abstract property"""  
    return self._myprop
```


Implementacja konkretnej klasy

Dziedziczy z ABC

```
class MyClass(MyABC):  
    """Implementation of MyABC"""
```

Standardowy konstruktor

```
def __init__(self, value=None):  
    self._my_prop = value
```

Implementacja
abstrakcyjnej metody

```
def do_something(self, value):  
    """Implementation of abstract method"""  
    self._myprop *= 2+value
```

```
@property  
def some_property(self):  
    """Implementation of abstract property"""  
    return self._myprop
```

Implementacja konkretnej klasy

Dziedziczy z ABC

```
class MyClass(MyABC):  
    """Implementation of MyABC"""
```

Standardowy konstruktor

```
def __init__(self, value=None):  
    self._my_prop = value
```

Implementacja
abstrakcyjnej metody

```
def do_something(self, value):  
    """Implementation of abstract method"""  
    self._myprop *= 2+value
```

Implementacja
abstrakcyjnego property

```
@property  
def some_property(self):  
    """Implementation of abstract property"""  
    return self._myprop
```

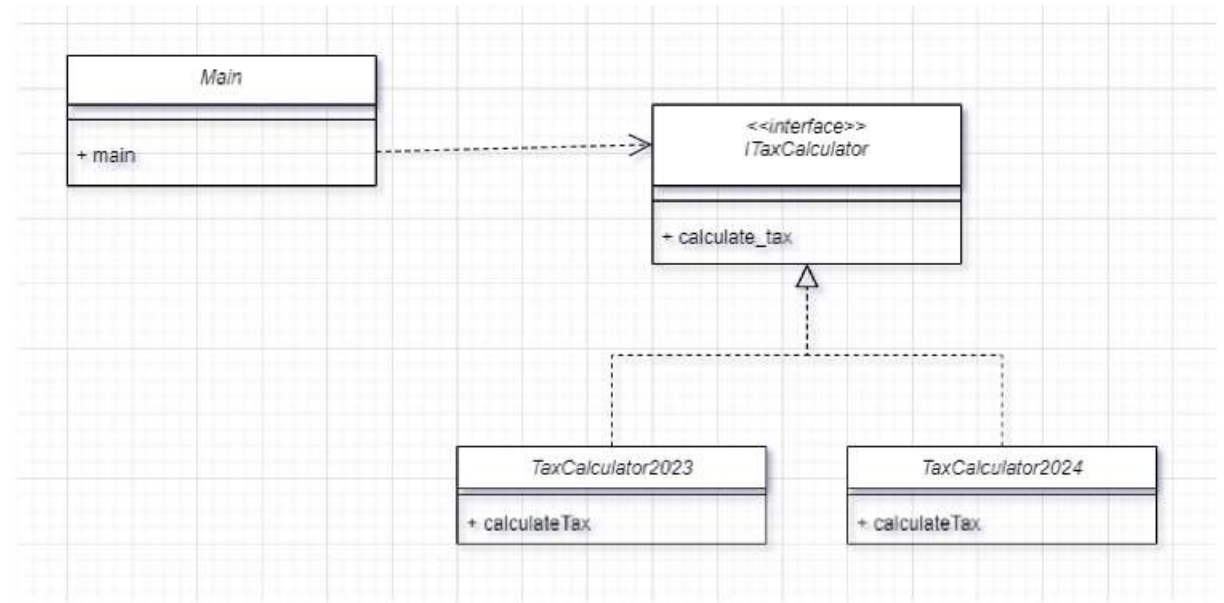

A co jeżeli **nie**
zaimplementujemy
abstrakcyjnej metody ?



A co jeżeli **nie**
zaimplementujemy
abstrakcyjnej metody ?

TypeError: Can't instantiate abstract class BadClass with
abstract methods do_something, some_property

Interfejs



Zasady projektowania

CRAIG LARMAN (1997)

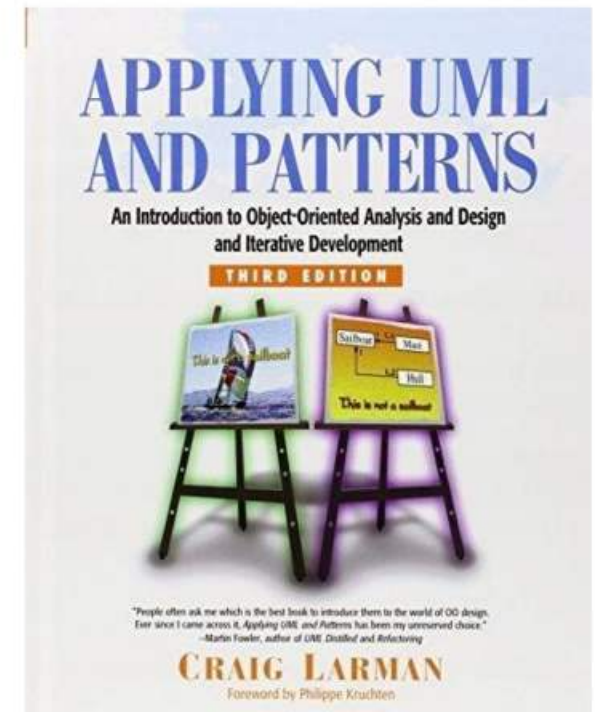
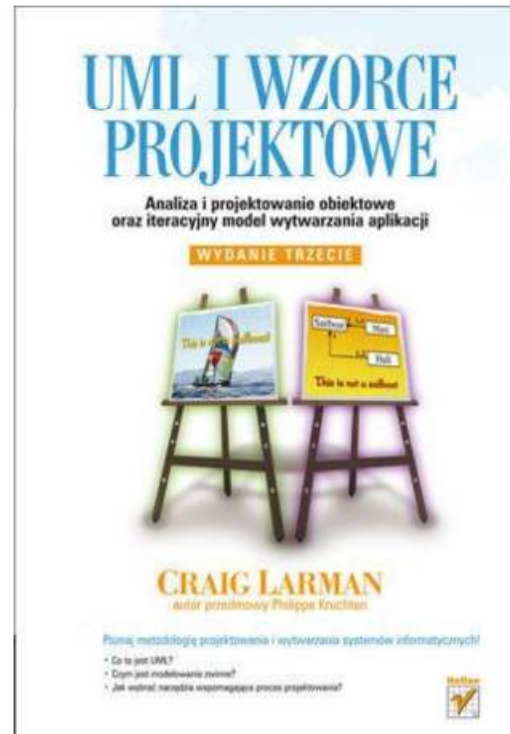
UML – Unified Modeling Language

OOA – Object Oriented Analysis

OOD – Object Oriented Design

OOP – Object Oriented Programming

GRASP - General Responsibility
Assignment Software Patterns



Obiektowe modelowanie dziedziny

OOA, OOD, OOP



OOA

Object Oriented Analysis



OOA

Analiza zorientowana obiektowo

Badanie i klasyfikację obiektów pojęciowych

OOA
zrób co należy



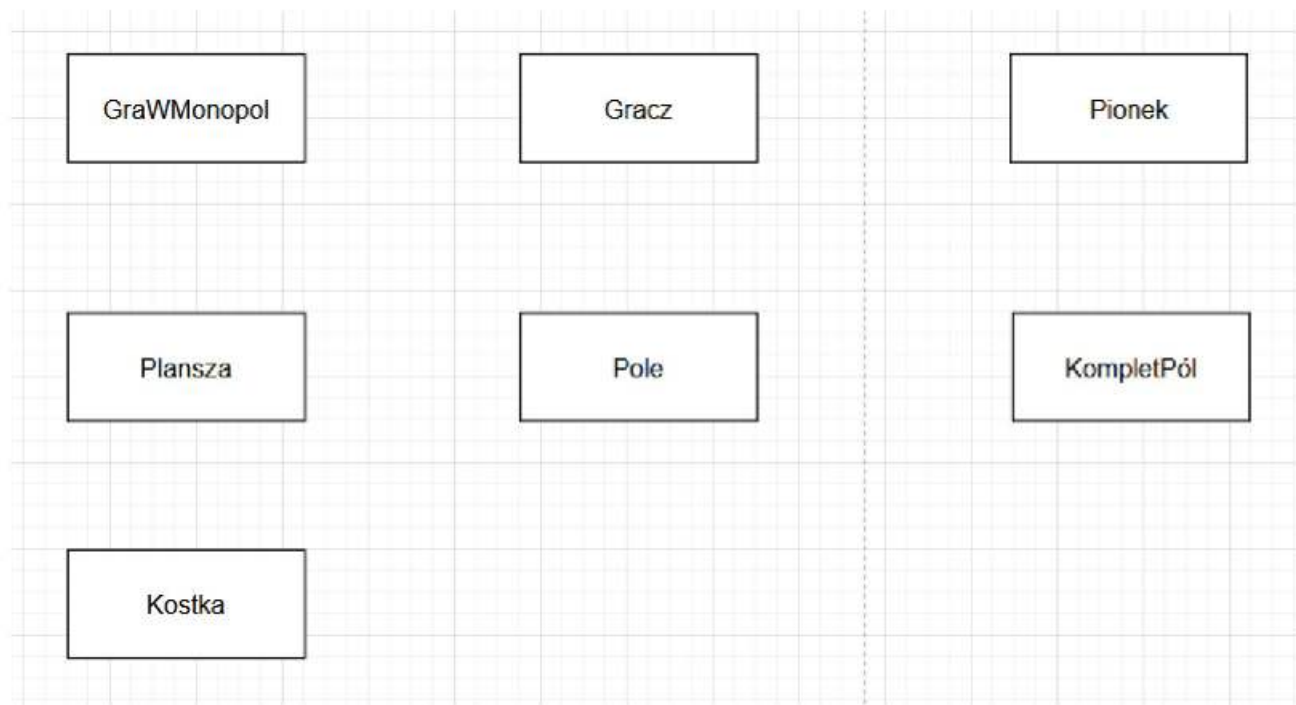
OOA
zrób co należy

◊ Opis słowny dziedziny

OOA
zrób co należy

- ◆ Opis słowny dziedziny
- ◆ Identyfikacja **klas pojęciowych** (lista kategorii klas pojęciowych lub analiza fraz rzeczownikowych)

Klasy pojęciowe



OOA
zrób co należy

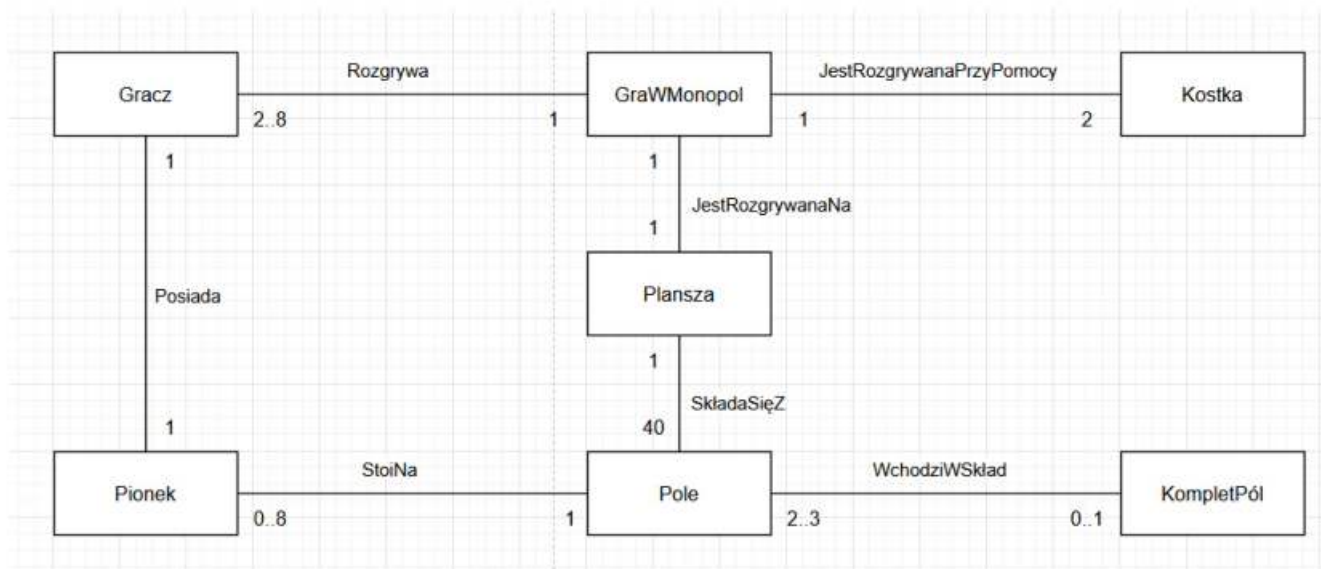
- ◊ Opis słowny dziedziny
- ◊ Identyfikacja **klas pojęciowych** (lista kategorii klas pojęciowych lub analiza fraz rzeczownikowych)
- ◊ Identyfikacja **powiązań** (lista kategorii powiązań, analiza fraz czasownikowych)

OOA

zrób co należy

- ◊ Opis słowny dziedziny
- ◊ Identyfikacja **klas pojęciowych** (lista kategorii klas pojęciowych lub analiza fraz rzeczownikowych)
- ◊ Identyfikacja **powiązań** (lista kategorii powiązań, analiza fraz czasownikowych)
- ◊ Określenie **liczebności** powiązań

Powiązania

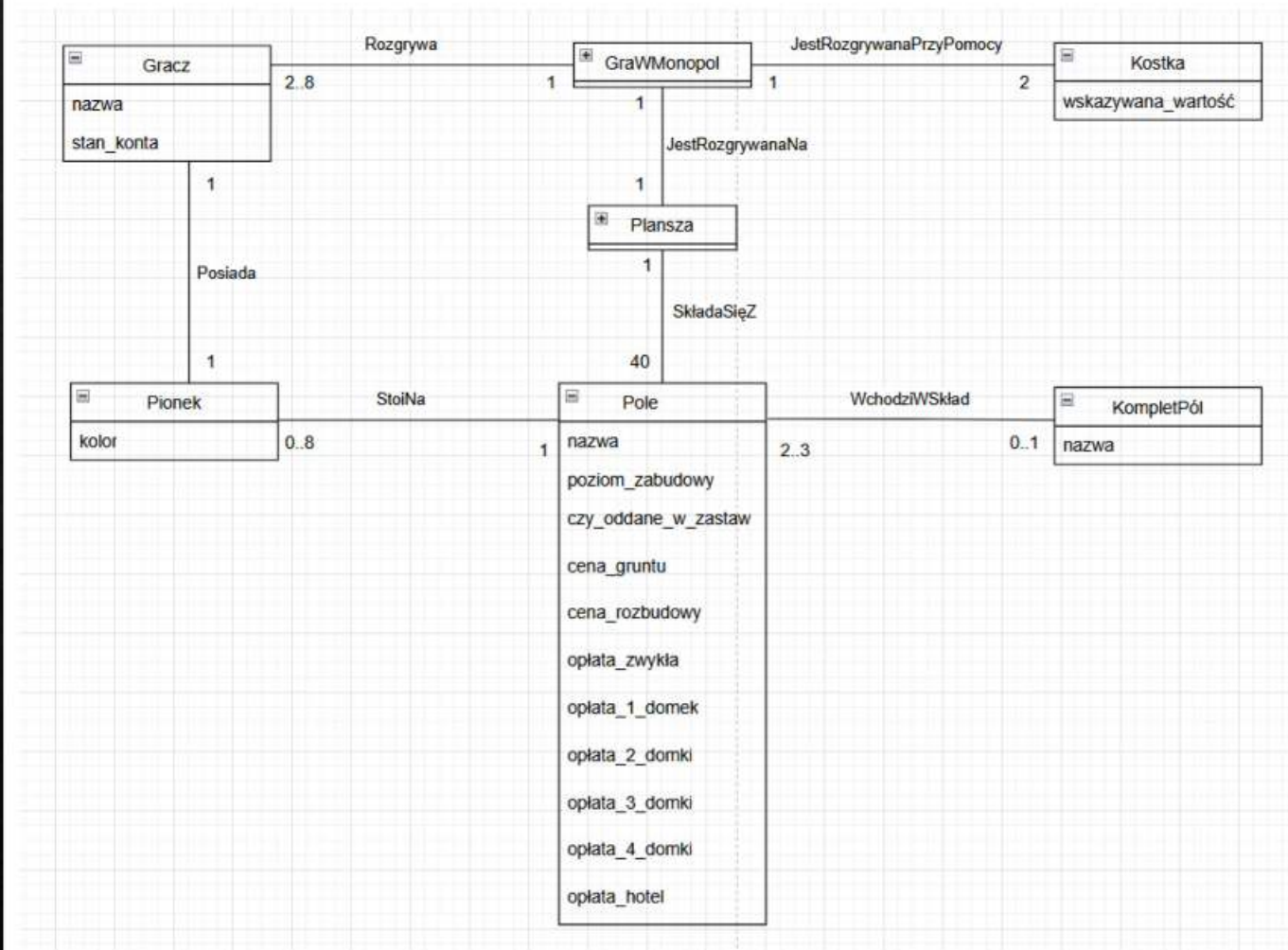


OOA

zrób co należy

- ◊ Opis słowny dziedziny
- ◊ Identyfikacja **klas pojęciowych** (lista kategorii klas pojęciowych lub analiza fraz rzeczownikowych)
- ◊ Identyfikacja **powiązań** (lista kategorii powiązań, analiza fraz czasownikowych)
- ◊ Uwzględnienie atrybutów klas pojęciowych

Atrybuty



Obiektowy model dziedziny

Obiektowy model dziedziny identyfikuje **klasy pojęciowe**, ich **atrybuty** i **powiązania** pomiędzy nimi. Stanowi bazę do tworzenia projektu obiektowego.

OOD

Object Oriented Design



OOD

Projektowanie zorientowane obiektowo

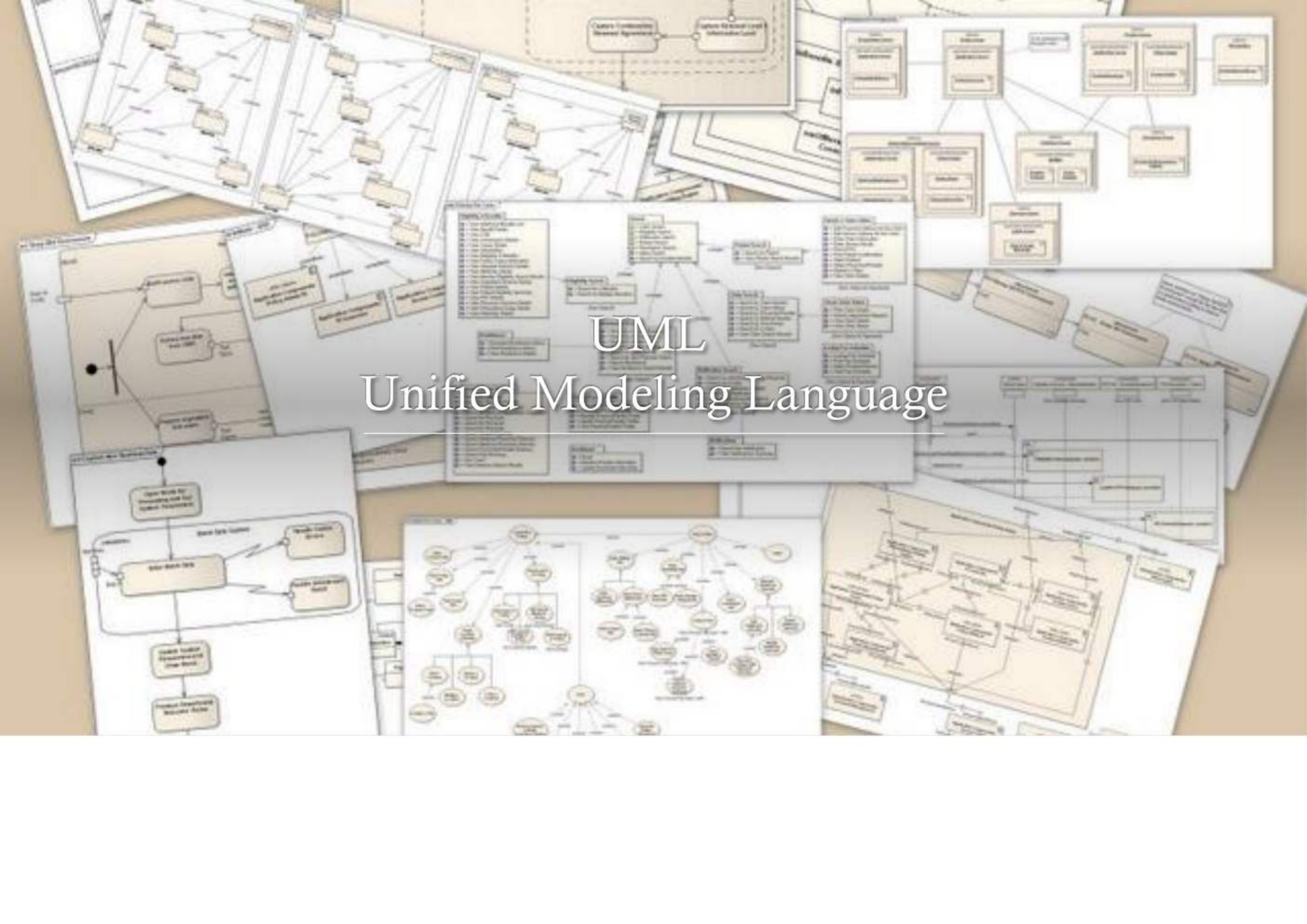
Opracowywanie **projektu obiektowego** na podstawie obiektowego modelu dziedziny. Polega na przypisaniu obiektom odpowiedzialności (za pamiętanie danych i za wykonywanie operacji na tych danych). To tutaj wykorzystywana jest wiedza o **wzorcach projektowych**.

OOD
zrób jak należy

- ◆ Opracowanie **klas projektowych** (zastosowanie wzorców projektowych)

OOD
zrób jak należy

◆ Nałożenie więzów



UML

Unified Modeling Language

UML

Zunifikowany język modelowania

Graficzny język modelowania.

Twórcami języka są: Grady Booch, Ivar Jacobson i James Rumbaugh. Pierwszy standard – UML 1.0 powstał w 1997. Dalszym rozwojem języka zajęła się grupa OMG.

Obecny standard - UML 2.5.1 wyróżnia 17 rodzajów diagramów (14 diagramów głównych i 3 abstrakcyjne).

Diagram klas

Class diagram

Prezentacja klas i zależności pomiędzy nimi.

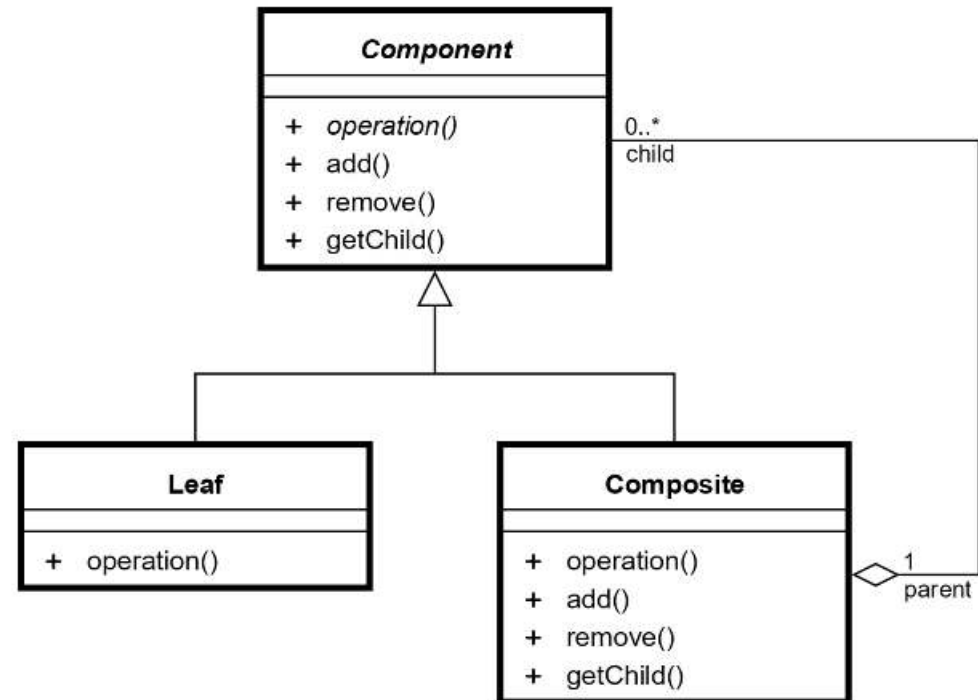


Diagram obiektów

UML

Object Diagram

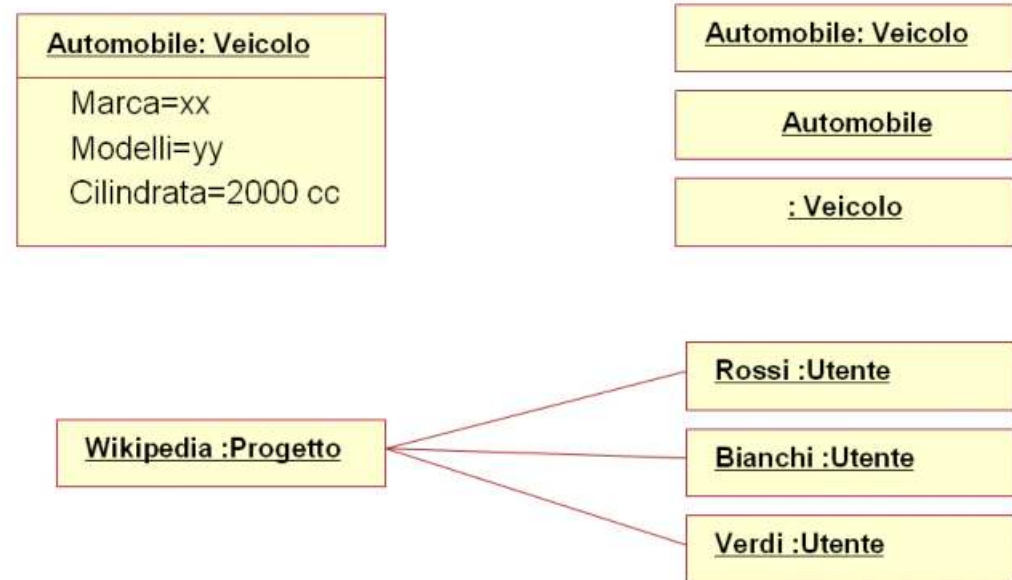


Diagram klas

Klasa

```
class Shape:
    def __init__(self, position_x, position_y):
        self.position_x = position_x
        self.position_y = position_y

    def render(self):
        ...
```

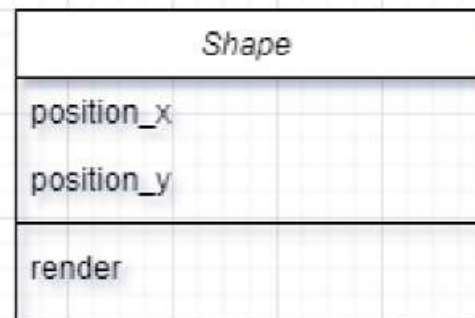


Diagram klas

Szczegóły klasy

```
class Person:
    def __init__(self, name: str, phone_number: str, age: int):
        self.name = name
        self._phone_number = phone_number
        self.__age = age

    def calculate_future_age(self, years: int) -> int:
        future_age = self.__age + years
        return future_age

    def _save_phone_number_to_file(self) -> None:
        ...

    def __update_age(self) -> None:
        ...
```

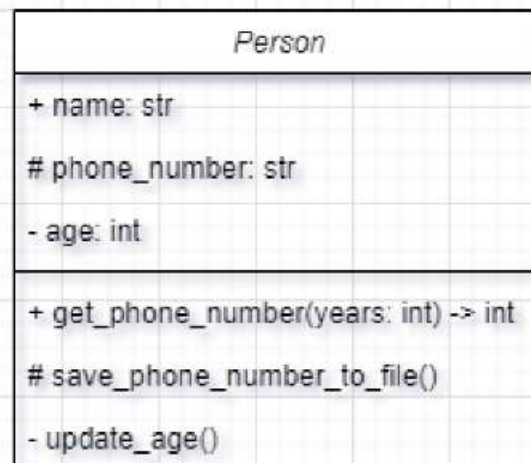


Diagram klas - relacje

Asocjacja *ang. association*



UML

Relacje

Uwaga!

Terminologia używana w modelowaniu obiektowym do opisu relacji pomiędzy klasami różni się miejscami od terminologii wprowadzonej w UML.

Terminologia wprowadzona w kolejnych slajdach jest charakterystyczna dla modelowania obiektowego.

Diagram klas - relacje

Dziedziczenie *ang. inheritance* (IS-A)

```
class Shape:  
    ...  
  
class Rectangle(Shape):  
    ...
```

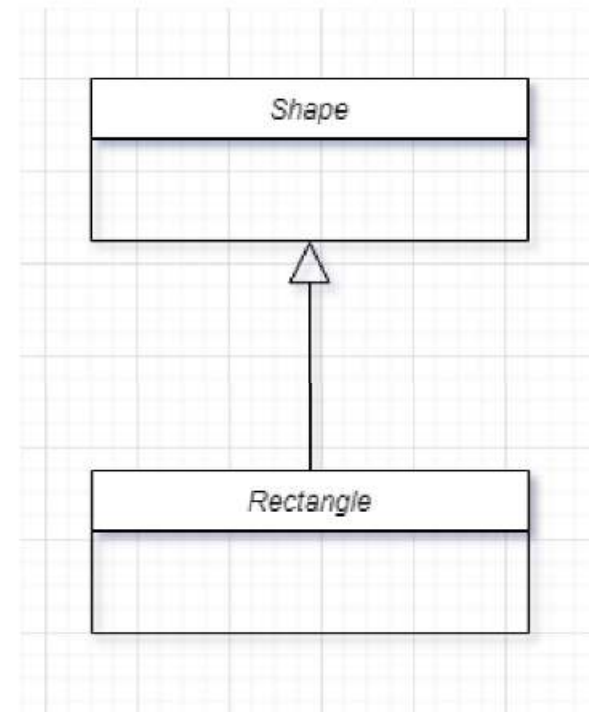


Diagram klas - relacje

Kompozycja *ang. composition* (OWNS-A)

```
class Size:
    ...

class Shape:
    def __init__(self):
        self.size = Size()
```

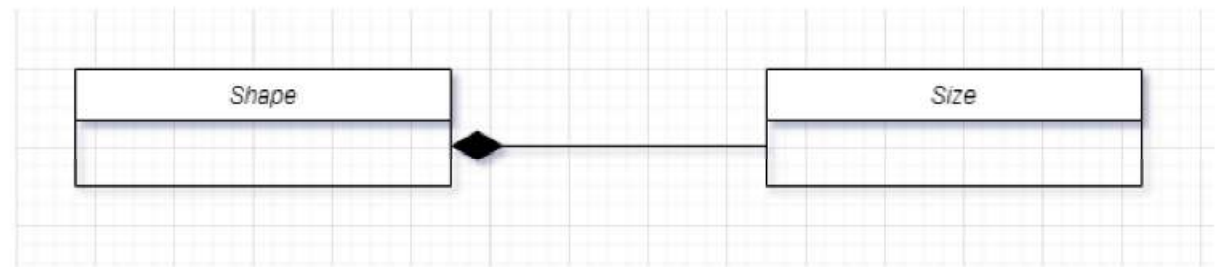


Diagram klas - relacje

Zależność ang. *dependency* (USE-A)

```
class Document:
```

```
...
```

```
class Shape:
```

```
    def render(self, doc: Document):
```

```
        ...
```

```
document = Document()
```

```
Shape().render(document)
```



UML

Relacje

Czasami w modelowaniu obiektowym wprowadza się jeszcze dwa typy relacji.

Diagram klas - relacje

Interfejs *ang. interface* (IMPLEMENT-A)

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):  
    @abstractmethod  
    def render(self):  
        ...
```

```
class Rectangle(Shape):  
    def render(self):  
        print("Rectangle")
```

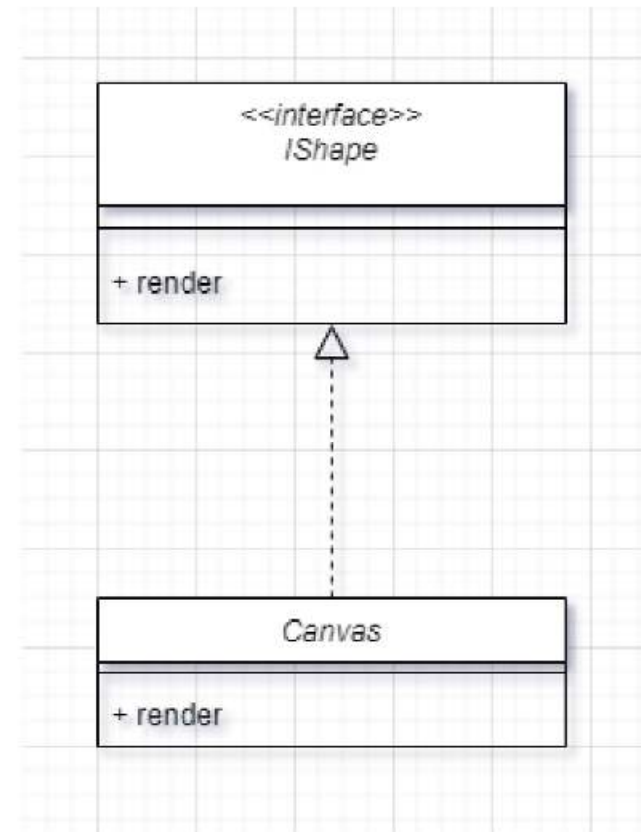


Diagram klas - relacje

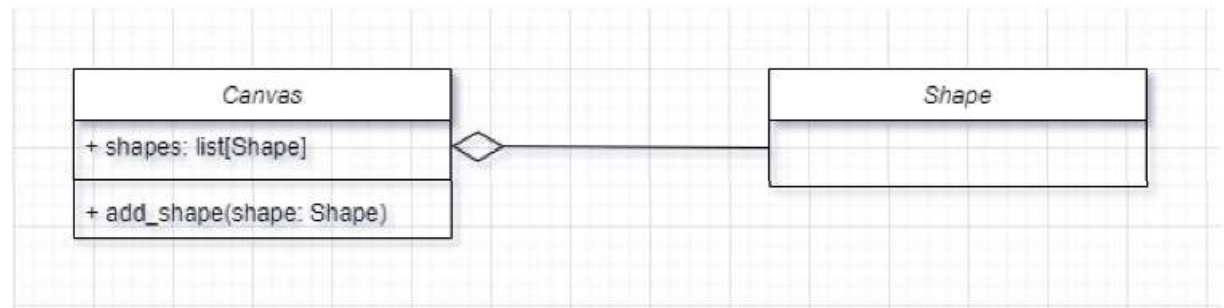
Agregacja *ang. aggregation* (HAS-A)

```
class Shape:
    ...

class Canvas:
    def __init__(self):
        self.shapes: list[Shape] = []

    def add_shape(self, shape: Shape):
        self.shapes.append(shape)

s = Shape()
canvas = Canvas()
canvas.add_shape(s)
```



OOP

Object Oriented Programming



OOP

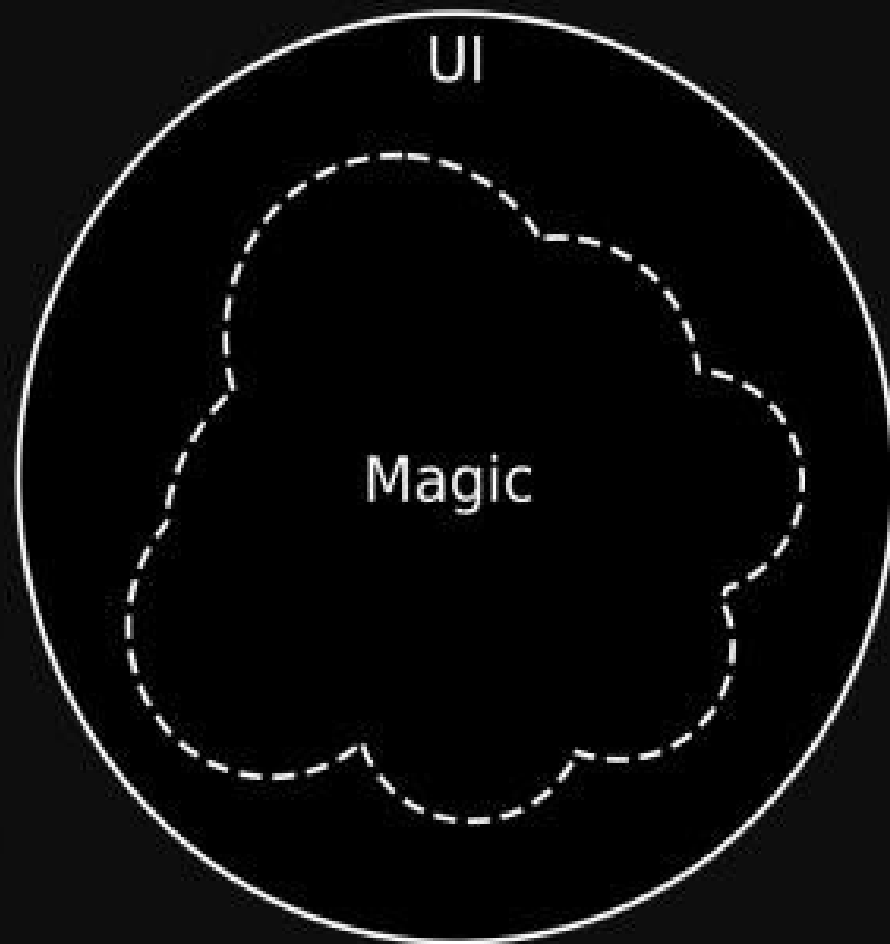
Programowanie zorientowane obiektowo

Implementacja klas projektowych w wybranym języku

Podstawowe zasady projektowania



What other see



What you see



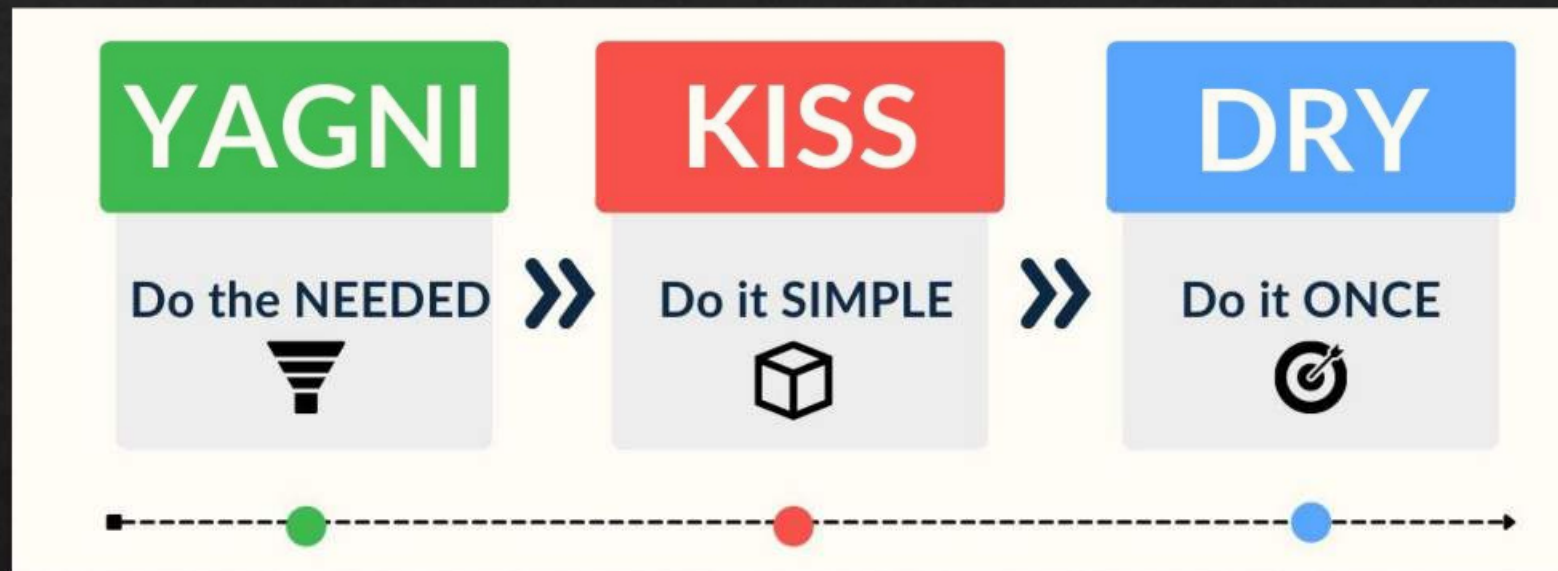


Projektowanie - podstawy

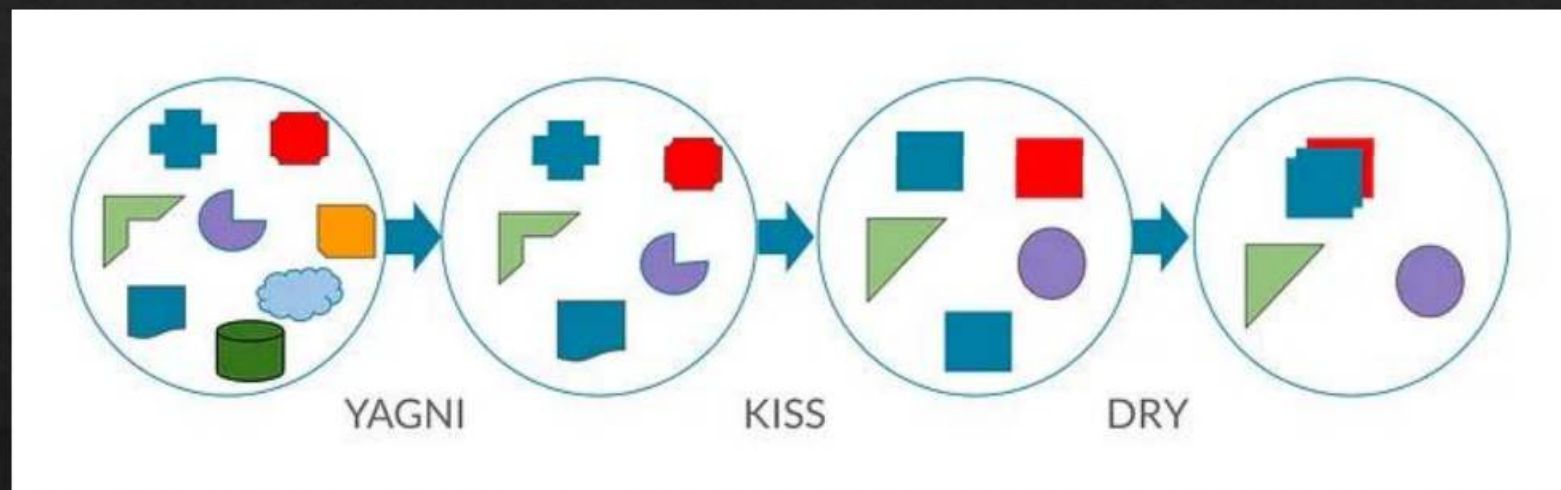
- SoC (Seperation of Concern)
Separacja zagadnień
- KISS (Keep It Simple, Stupid)
- YAGNI (You Aren't Gonna Need It)
 - MoSCoW (Must have, Should have, Could have, Won't have)

Podstawowe zasady programowania





Suche pocalunki Jagny



Suche pocatunki Jagny

Programowanie - podstawy

- SoC (Seperation of Concern)
 - Separacja zagadnień
- DRY (Don't Repeat Yourself)
 - RoT (Rule of Three)
- KISS (Keep It Simple, Stupid)
- YAGNI (You Aren't Gonna Need It)
 - DTSTTCPW (DoTheSimplestThingThatCouldPossiblyWork)

Programowanie - podstawy

- SoC (Seperation of Concern)
Separacja zagadnień
- DRY (Don't Repeat Yourself)
 - RoT (Rule of Three)
- KISS (Keep It Simple, Stupid)
- YAGNI (You Aren't Gonna Need It)
 - DTSTTCPW (DoTheSimplestThingThatCouldPossiblyWork)
- Demeter Law

Programowanie w Pythonie - podstawy

```
>>> import this
```

```
The Zen of Python, by Tim Peters
```

```
Beautiful is better than ugly.
```

```
Explicit is better than implicit.
```

```
Simple is better than complex.
```

```
Complex is better than complicated.
```

```
Flat is better than nested.
```

```
Sparse is better than dense.
```

```
Readability counts.
```

```
Special cases aren't special enough to break the rules.
```

```
Although practicality beats purity.
```

```
Errors should never pass silently.
```

```
Unless explicitly silenced.
```

```
In the face of ambiguity, refuse the temptation to guess.
```

```
There should be one-- and preferably only one --obvious way to do it.
```

```
Although that way may not be obvious at first unless you're Dutch.
```

```
Now is better than never.
```

```
Although never is often better than *right* now.
```

```
If the implementation is hard to explain, it's a bad idea.
```

```
If the implementation is easy to explain, it may be a good idea.
```

```
Namespaces are one honking great idea -- let's do more of those!
```

```
>>> █
```

GRASP

General Responsibility Assignment Software Patterns

Zasady GRASP

- **High cohesion**
- **Low couplings**
- Information expert
- Pure fabrication
- Creator
- Controller
- Indirection
- Polymorphism
- Protected variations

Cohesion

Spójność

Spójność to miara dotycząca wewnętrznej struktury **komponentu oprogramowania**, czymkolwiek ten komponent oprogramowania jest (pakietem, modulem, klasą, funkcją, ...).

Komponent o wysokiej spójności wykonuje jedno, precyzyjnie określone zadanie. **Komponent o niskiej spójności** to **komponent**, który realizuje wiele, niepowiązanych ze sobą zadań.


```
def handle_stuff(d: Data, quantity: int, screenX: int,
                screenY: int, status: int, c: Color):

    update_corporate_datebase(d, q, status)
    for i in range(0, quantity):
        profit[i] = revenue[i] - expense[i] * status
    new_color = c
    status = SUCCESS
    display_profits(screenX, screenY, status, d, new_color)
```

Funkcja o niskiej spójności

Cohesion

Spójność

Niska spójność świadczy o nieprawidłowym zaprojektowaniu **komponentu** i jego wysokiej złożoności. Taki komponent należy rozbić na kilka mniejszych komponentów. Po jednym **komponencie** na każde zadanie (odpowiedzialność).

W przypadku klas, spójność **klasy** oceniamy sprawdzając, czy metody **klasy** pracują na jednym i tym samym zbiorze atrybutów.

Cohesion

Spójność

Wyróżniamy 7 typów spójności:

- functional cohesion
- sequential cohesion
- communicational cohesion
- procedural cohesion
- temporal cohesion
- logical cohesion
- coincidental cohesion

Coupling

Sprzężenia

Sprzężenie - miara stopnia zależności modułów od siebie nawzajem.

Coupling

Sprzężenia

Wyróżniamy 5 typów sprzężeń:

- data coupling
- control coupling
- external coupling
- common coupling
- content coupling




```
def check_email_security(email):  
    if email.header.bearer.invalid():  
        raise Exception("Email header bearer is invalid")  
    elif email.header.received != email.header.received_spf:  
        raise Exception("Received mismatch")  
    else:  
        print("Email header is secure")
```

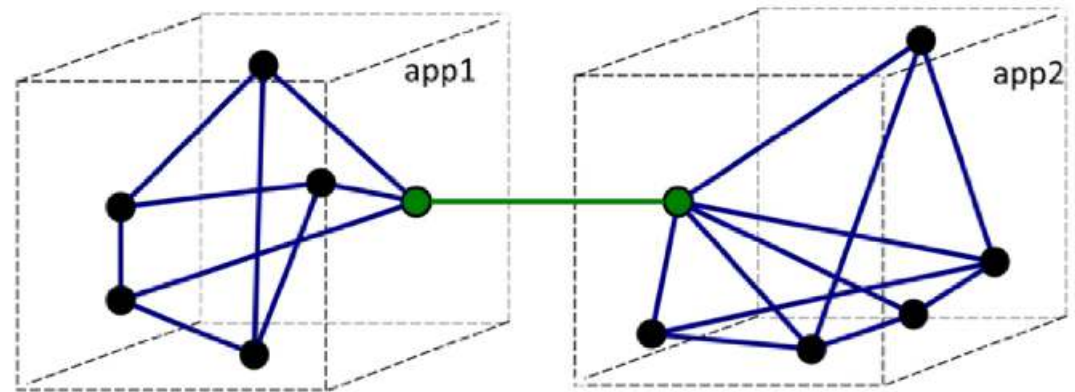
Funkcja o silnych sprzężeniach

*W kodzie staramy się utrzymać wysoką spójność i luźne
sprzężenia*

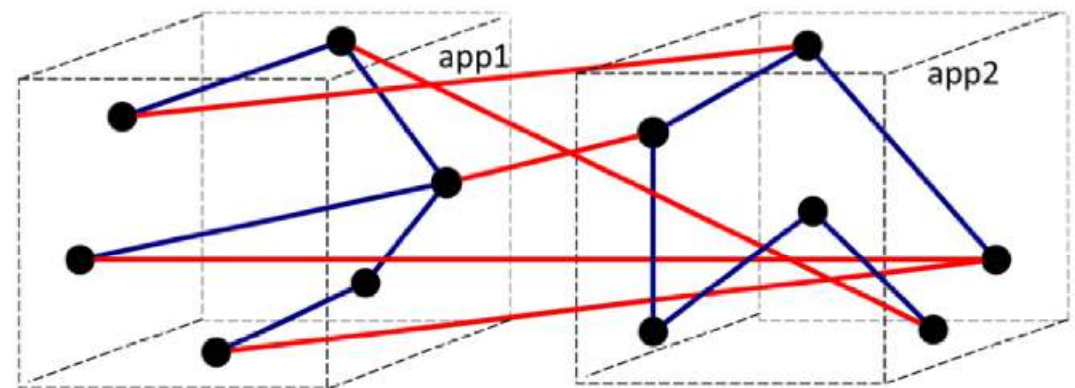
W kodzie staramy się utrzymać wysoką spójność i luźne sprzężenia

Klasy o wysokiej spójności jest znacznie łatwiej utrzymywać. Poza tym klasa, której jesteśmy w stanie nadać jasno określoną odpowiedzialność jest łatwiejsza do zrozumienia.

Wysoka spójność
(high cohesion)

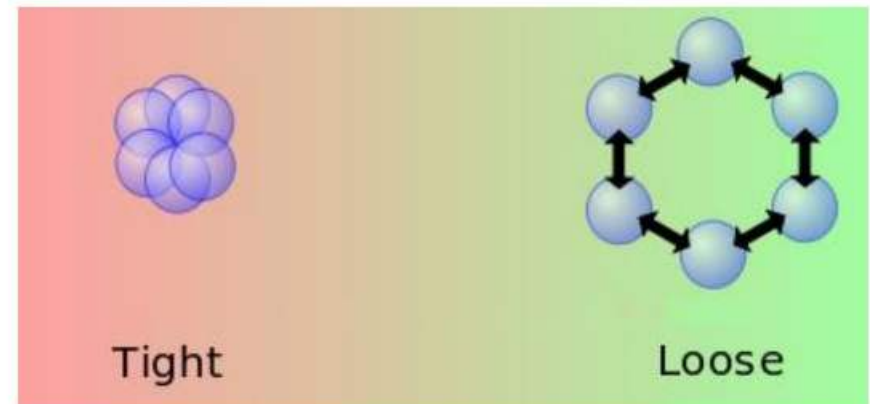


a) Good



b) Bad

Luźne sprzężenia
(loose coupling)

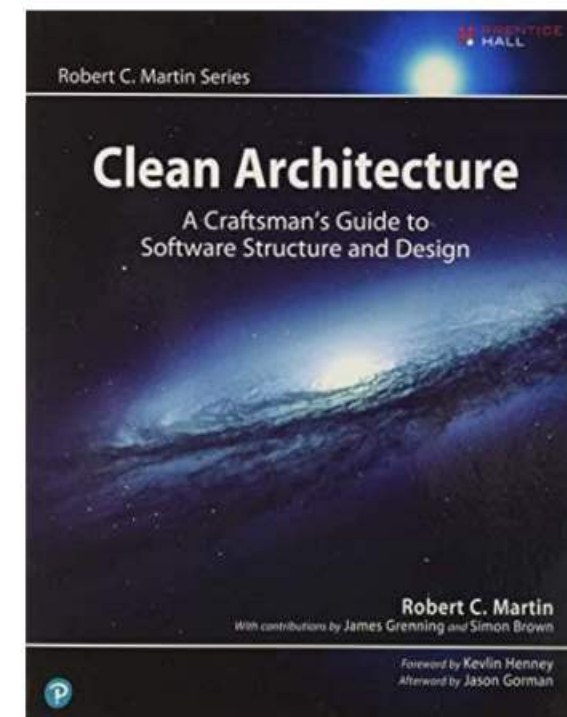


SOLID Principles of Object Oriented Design

Robert C. Martin (2000 r)

Robert C. Martin (Uncle Bob)

Opublikowana w 2017



SOLID Principles of Object Oriented Design

(sformułowane w 2000r. w artykule Design Principles and Design Patterns)

S

Reguła jednej odpowiedzialności

SRP (Single Responsibility Principle)

S – Single Responsibility Principle



Każdy komponent powinien mieć jedną i tylko jedną przyczynę zmiany.

Klasa powinna mieć tylko jedną odpowiedzialność, czyli albo gotowanie, albo zmywanie, ale nigdy to i to.

SOLID Principles of Object Oriented Design

S

Reguła jednej odpowiedzialności

SRP (Single Responsibility Principle)

O

Reguła otwarte-zamknięte

OCP (Open-Close Principle)

O - Open Close Principle

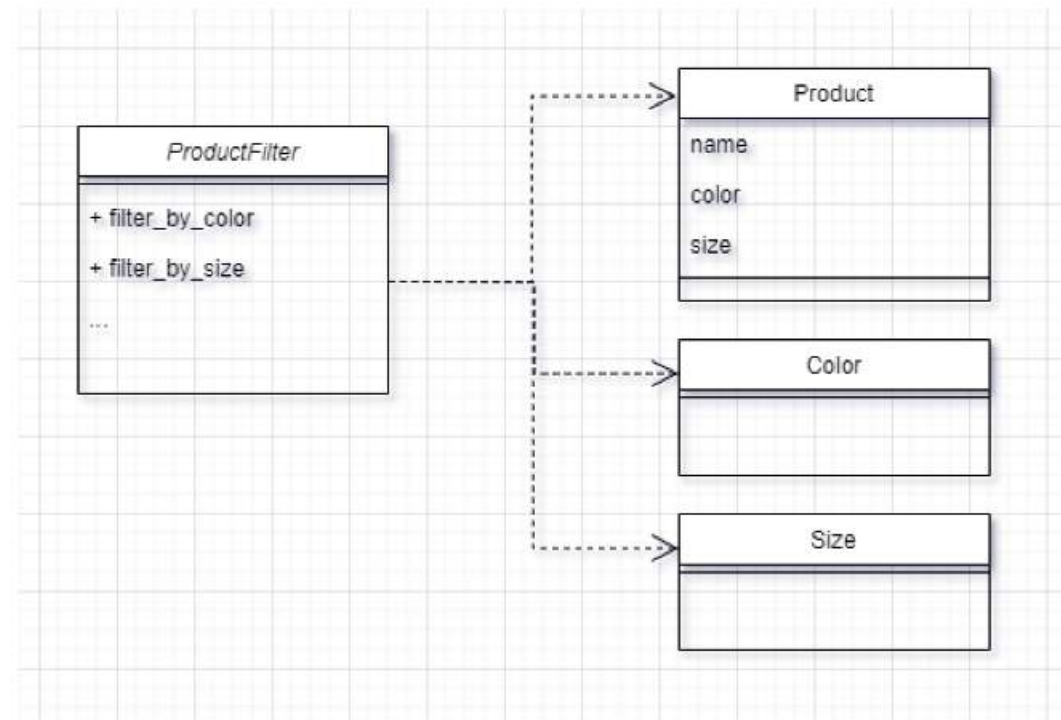


Element oprogramowania powinien być otwarty na rozbudowę, a zamknięty na modyfikacje.

Klasa powinna być otwarta na rozbudowę (przeważnie poprzez zastosowanie dziedziczenia), ale zamknięta na modyfikacje.

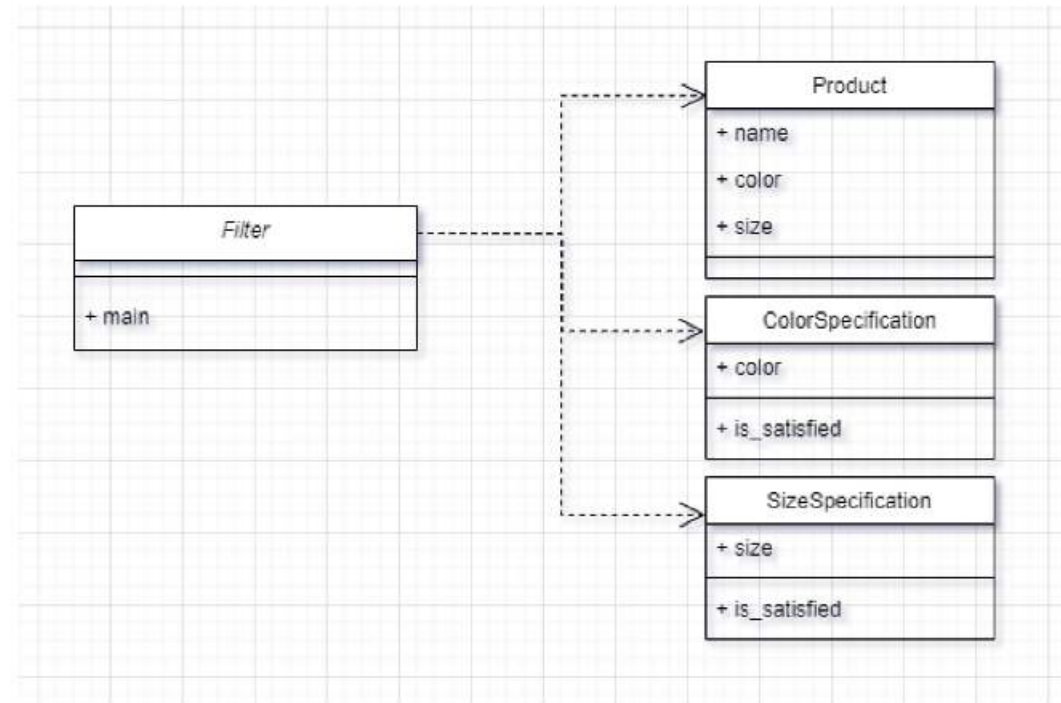
OPC

Open Close Principle - before



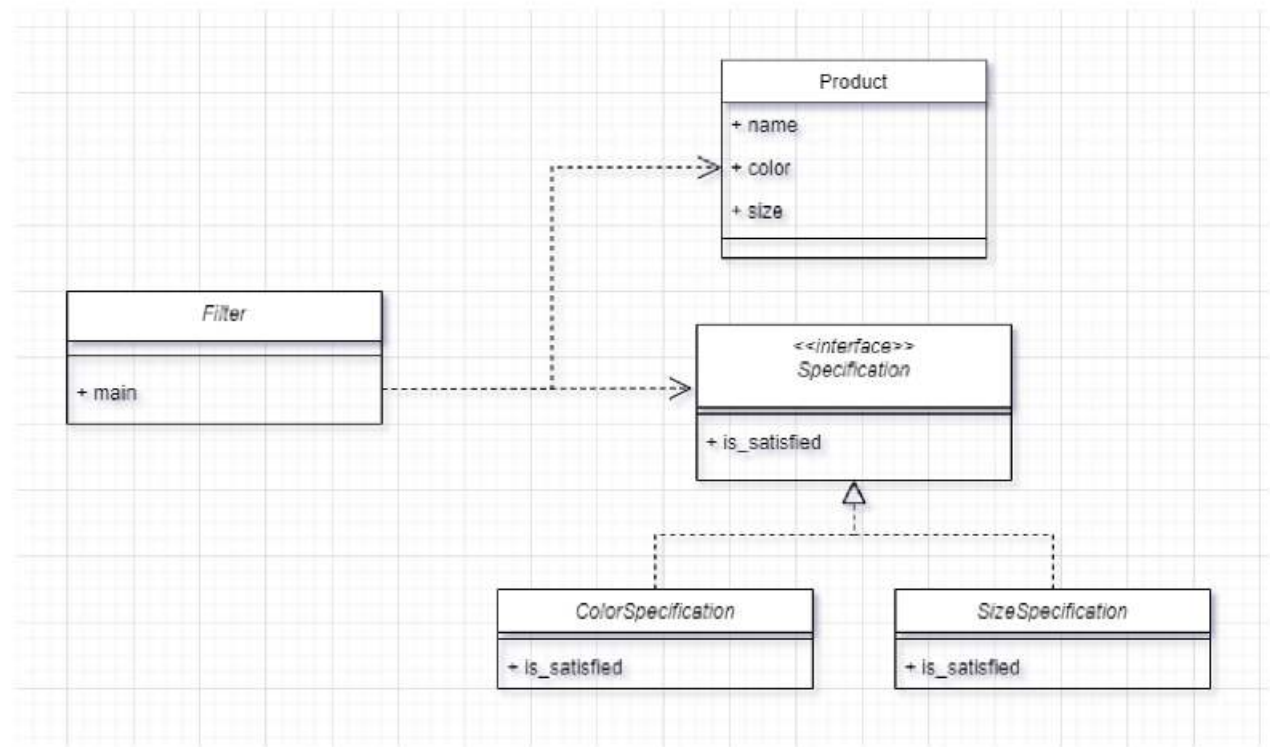
OPC

Open Close Principle - after



OPC

Open Close Principle – after 2



SOLID Principles of Object Oriented Design

S

Reguła jednej odpowiedzialności

SRP (Single Responsibility Principle)

O

Reguła otwarte-zamknięte

OCP (Open-Close Principle)

L

Zasada podstawień Barbary
Liskov

LSP (Liskov Substitution Principle)

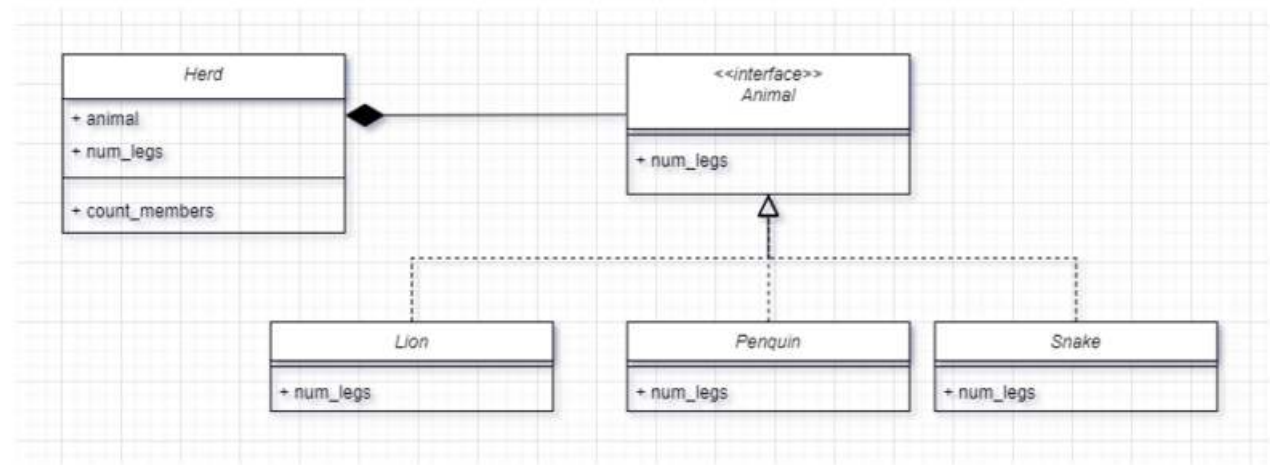
L – Liskov Substitution Principle



Podklasy powinny być w stanie zastąpić swoich rodziców bez wywoływania błędu w programie.

LSP

Liskov Substitution Principle



SOLID Principles of Object Oriented Design

S

Reguła jednej odpowiedzialności

SRP (Single Responsibility Principle)

O

Reguła otwarte-zamknięte

OCP (Open-Close Principle)

L

Zasada podstawień Barbary
Liskov

LSP (Liskov Substitution Principle)

I

Zasada rozdzielania interfejsów

ISP (Interface Segregation Principle)

I – Interface Segregation Principle



Wiele różnych interfejsów jest lepsze niż jeden interfejs typu do-it-all

SOLID Principles of Object Oriented Design

S

Reguła jednej odpowiedzialności

SRP (Single Responsibility Principle)

O

Reguła otwarte-zamknięte

OCP (Open-Close Principle)

L

Zasada podstawień Barbary
Liskov

LSP (Liskov Substitution Principle)

I

Zasada rozdzielania interfejsów

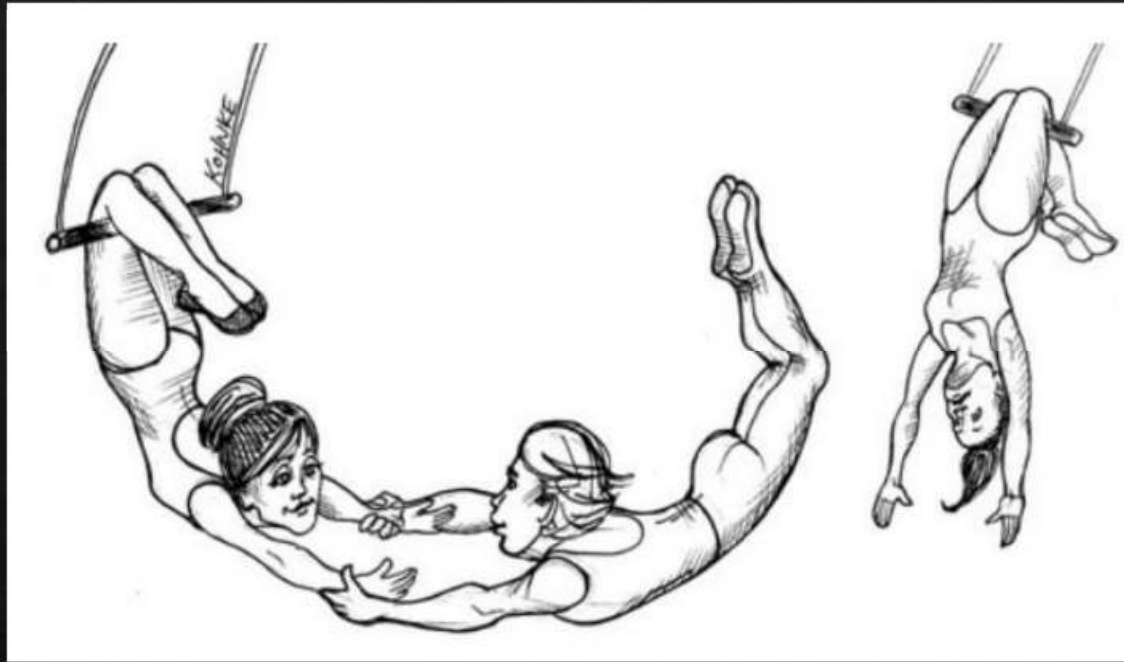
ISP (Interface Segregation Principle)

D

Zasada odwrócenia zależności

DIP (Dependency Inversion Principle)

D – Dependency Inversion Principle



Powinniśmy programować za pomocą abstrakcji, nie implementacji. Implementacje mogą się zmieniać, abstrakcje nie powinny

DIP

Dependency Inversion Principle - before

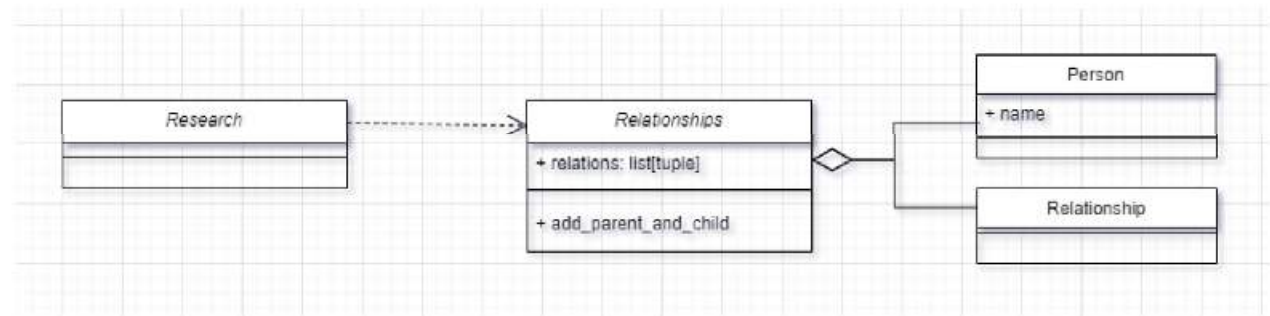
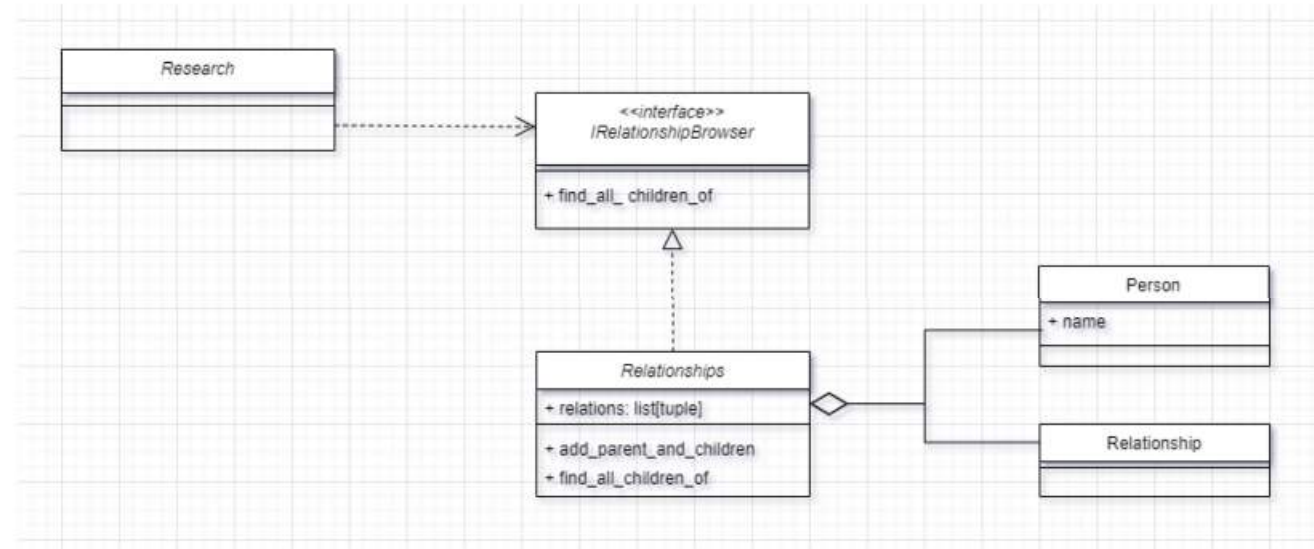


Diagram klas - relacje

Asocjacja ang. *association*



SOLID Principles of Object Oriented Design

S

Reguła jednej
odpowiedzialności

SRP (Single Responsibility Principle)

O

Reguła otwarte-zamknięte

OCP (Open-Close Principle)

L

Zasada podstawień Barbary
Liskov

LSP (Liskov Substitution Principle)

I

Zasada rozdzielania interfejsów

ISP (Interface Segregation Principle)

D

Zasada odwrócenia zależności

DIP (Dependency Inversion Principle)

STUPID - antySOLID

??? (??? r)

STUPID

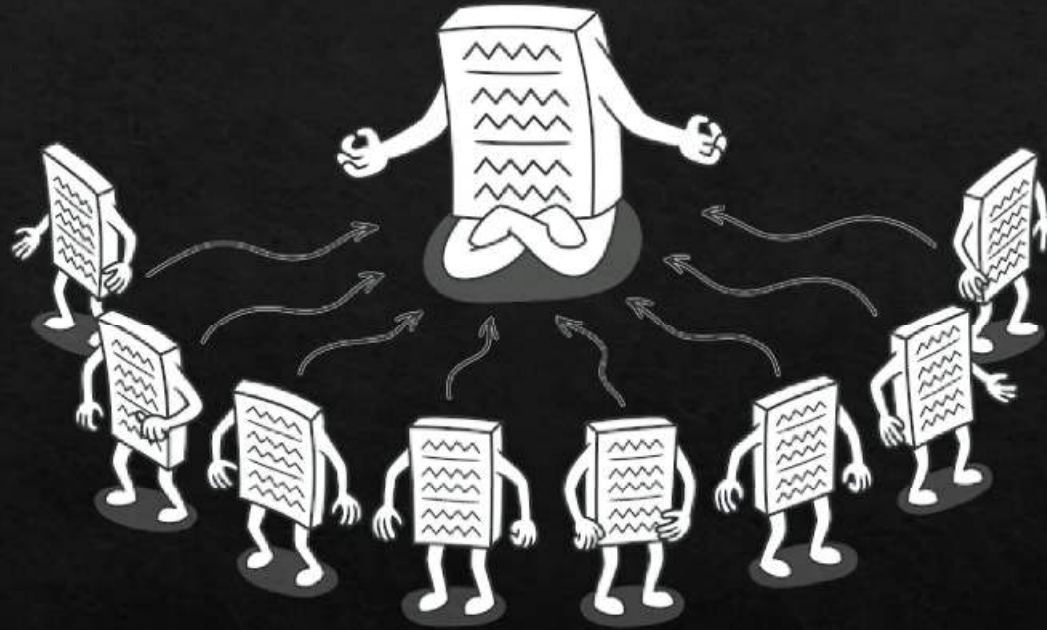
Zestaw 5 antywzorców na wzór SOLID

- **S**ingleton
- **T**ight coupling
- **U**ntestability
- **P**remature optimizations
- **I**ndescriptive naming

STUPID, czasami przedstawiany jest jako zestaw antywzorców przed, którym chroni stosowanie reguł SOLID.

Chociaż takie twierdzenie nie jest bezpodstawne, w ogólności jest nadużyciem.

S – Singleton



Stany o zasięgu globalnym przeważnie są trudne do przetestowania.

T– Tight coupling (silne sprzeżenia)



Za duzo sprzezen

U – Untestability



Testowanie nie powinno być trudne

P – Premature optimizations

(przedwczesne optymalizacje)



"Premature optimization is the root of all evil"

Donald Knuth

I – Indescriptive naming



Używaj precyzyjnych, samodefiniujących nazw zmiennych/funkcji/klas. Na ile to możliwe startaj się unikać skrótowców.

D – Duplication



Don't Repeat Yourself!
Keep it simple, Stupid!

Be lazy the right way – write code only once

CUPID

Dan North (2017 r)

*SOLID spotkał się ze sporą krytyką w ostatnich 10 latach.
Najczęściej podnoszonym zarzutem jest wiek tych zasad.
30 lat temu programiści mierzyli się z zupełnie innymi
zagadnieniami niż dzisiaj.*

*Dan North zakwestionował zasady SOLID w 2017
roku poprzez zaproponowanie nowych zasad - CUPID*

CUPID

Zestaw 5 wzorców stworzonych jako krytyka SOLID

- **C**omposable – plays well with others
- **U**nix philosophy – does one thing well
- **P**redictable – does what you expect
- **I**diomatic – feels natural
- **D**omain-based

Composable

Plays well with others

Composable code has classes and functions that are easily combined, assembled in different combinations, to address specific requirements.

Unix philosophy

Does one thing well

Unix is a longstanding operating system (or family of systems) that led to today's Linux. The Unix philosophy encourages coding components that work well together, each doing one thing and doing it well.

Predictable

Does what you expect

The goal here is code that is robust, reliable, and resilient. It should also behave as expected.

Idiomatic

Feels natural

With CUPID, the code should be understandable by someone else, not just you, the original coder.

Domain-based

The solution domain models the problem domain in language and structure

The code should be compatible with its domain, using the problem domain's language to avoid the need for future developers to 'translations' or cognitive leaps in order to understand it.

Wzorce projektowe



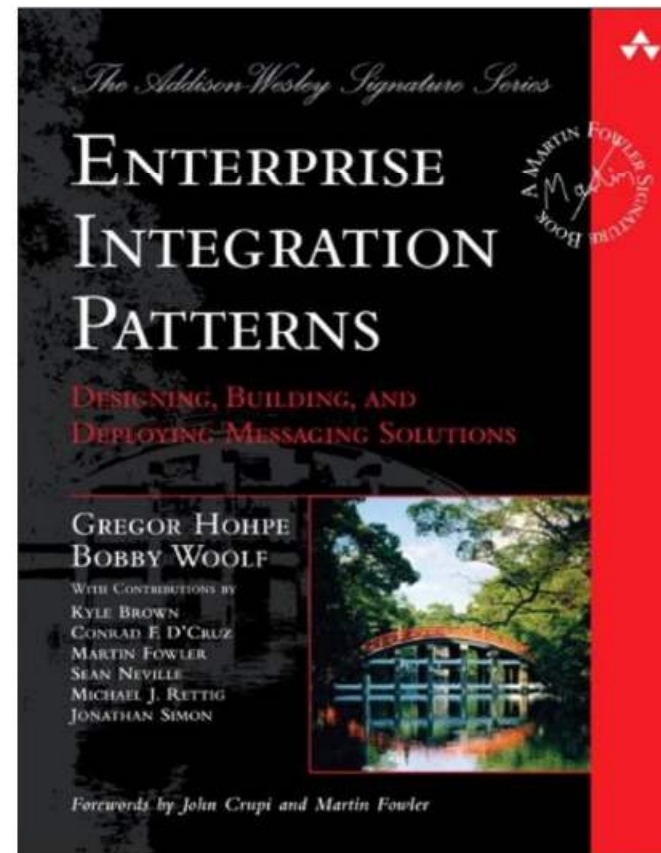
Wzorce projektowe

- **GoF (Gang of Four)**
Klasyczny zestaw zawierający 23 wzorce projektowe
- **EIP (Enterprise Integration Patterns)**
Zestaw 64 wzorców projektowych związanych z integracją systemów i integracją międzykomponentową
- **PoEAA (Patterns of Enterprise Application Architecture)**
Zestaw około 40 wzorców projektowych (liczba może się różnić pomiędzy wydaniem) opisanych przez Martina Flowera
- **Cloud Desing Patterns**
Wzorce projektowe dostosowane do tworzenia aplikacji w chmurze
- **Concurrency Patterns**
Wzorce projektowe dostosowane do tworzenia aplikacji współbieżnych
- **Security Patterns**
Wzorce projektowe związane z aspektami bezpieczeństwa
- **Game Programming Patterns**
Wzorce projektowe związane z programowaniem gier komputerowych
- **Test Design Patterns**
Wzorce projektowe związane z projektowaniem testów oprogramowania
- **J2EE Patterns**
Wzorce projektowe stosowane w ramach platformy Java 2 Enterprise Edition
- **JSP Patterns**
Zestaw wzorców projektowych związanych z językiem JavaScript

Gregor Hohpe, Bobby Woolf

Opublikowana w 2003

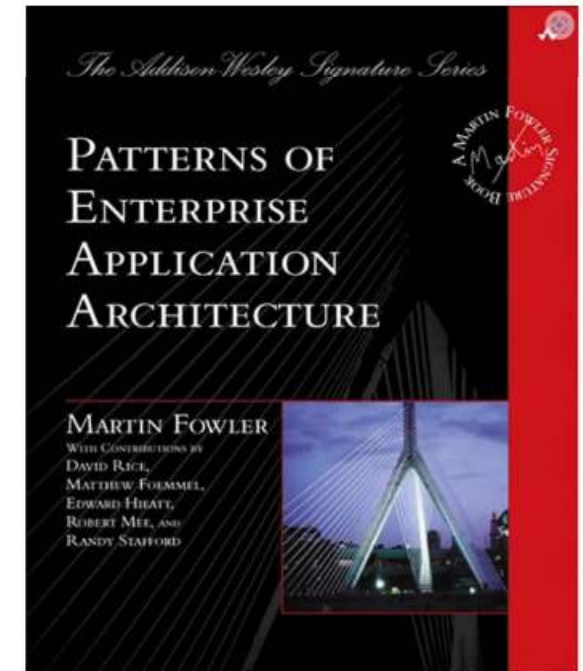
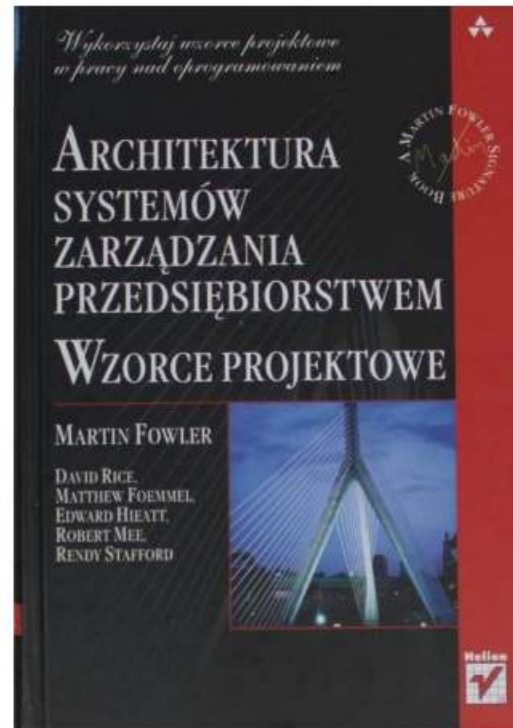
Zawiera zestaw 65 wzorców
pogrupowanych w 9 kategorii.



Martin Fowler

Opublikowana w 2003

Stanowi ogólny przegląd dziedziny. W treści nawiązuje i analizuje wiele różnych wzorców bez prób ich kategoryzowania.



GAMMA, HELM, JOHNSON, VLISSIDES (GoF – Gang of Four)

Opublikowana w 1995

Pierwsza publikacja dotycząca wzorców projektowych zawierająca całościowe podejście do zagadnienia. Zawiera zestaw 23 wzorców pogrupowanych w 3 kategorie.



Podstawowa klasyfikacja wzorców projektowych (kategoryzacja Gamma)



Podstawowa klasyfikacja wzorców projektowych (kategoryzacja Gamma)

Kreacyjne

Tworzenie obiektów

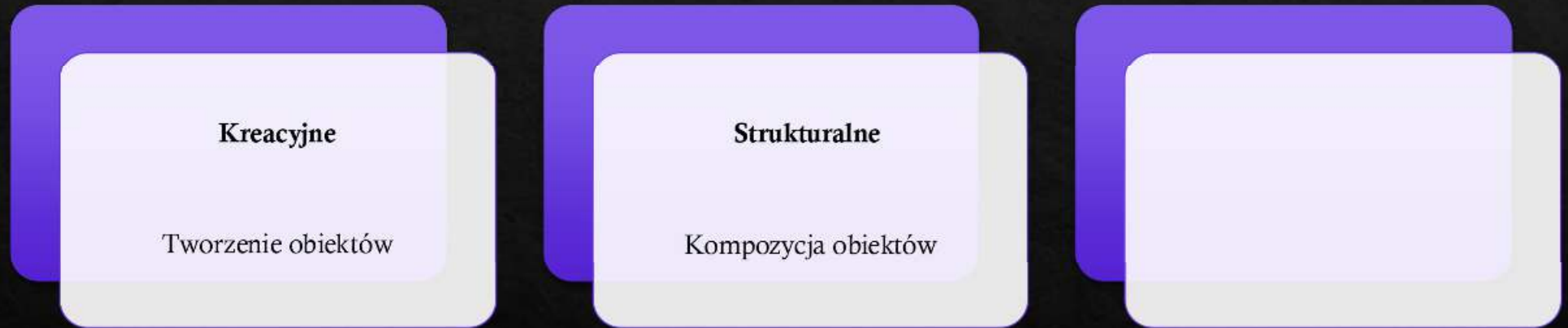
Podstawowa klasyfikacja wzorców projektowych (kategoryzacja Gamma)

Kreacyjne

Tworzenie obiektów

Strukturalne

Kompozycja obiektów



Podstawowa klasyfikacja wzorców projektowych (kategoryzacja Gamma)

Kreacyjne

Tworzenie obiektów

Strukturalne

Kompozycja obiektów

Behawioralne (czynnościowe)

Interakcja
pomiędzy obiektami,
odpowiedzialności

Alternatywna klasyfikacja wzorców projektowych



Alternatywna klasyfikacja wzorców projektowych



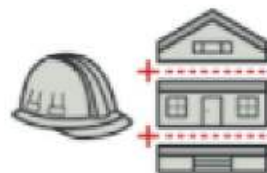
Alternatywna klasyfikacja wzorców projektowych



Wzorce konstrukcyjne	Wzorce strukturalne	Wzorce behawioralne
- Budowniczy - Fabryka abstrakcyjna - Metoda wytwórcza - Prototyp - Singleton	- Adapter - Dekorator - Fasada - Kompozyt - Most - Proxy - Pylek	- Interpreter - Iterator - Łańcuch zobowiązań - Mediator - Metoda szablonowa - Obserwator - Odwiedzający - Pamiętka - Polecenie - Stan - Strategia

Wzorce kreacyjne

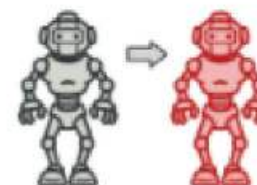
(Creational Patterns)



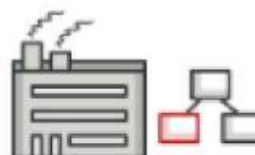
Budowniczy
Builder



Fabryka abstrakcyjna
Abstract Factory



Prototyp
Prototype



**Metoda
wytwórcza**
Factory Method

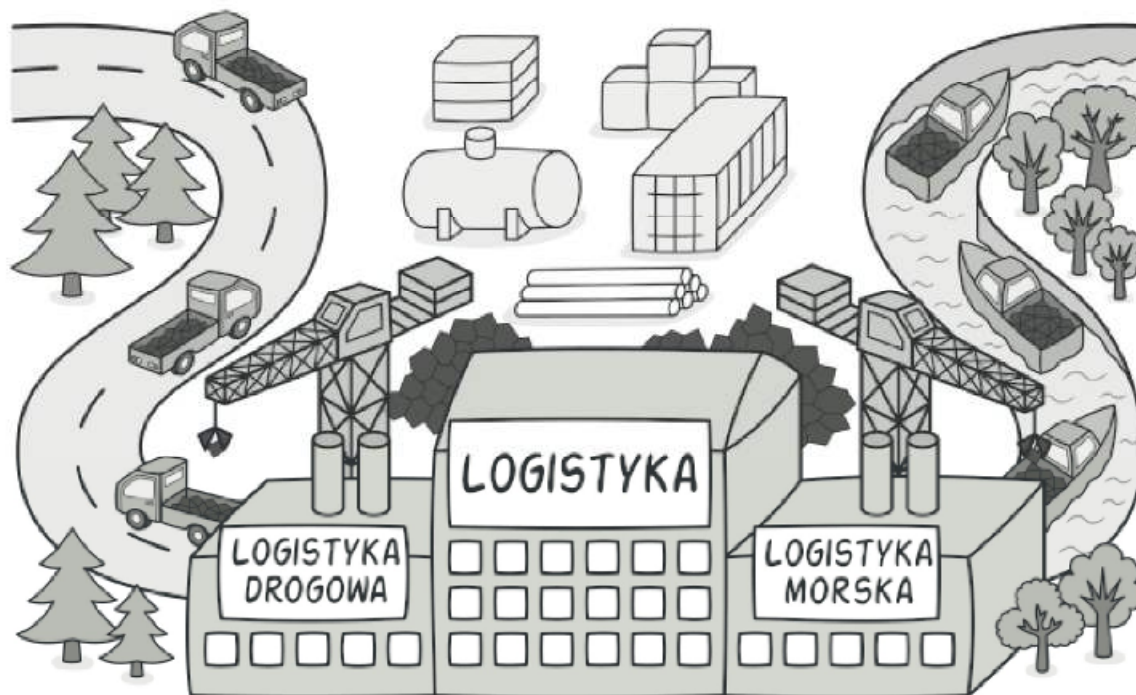


Singleton
Singleton

Metoda wytwórcza (Factory method)

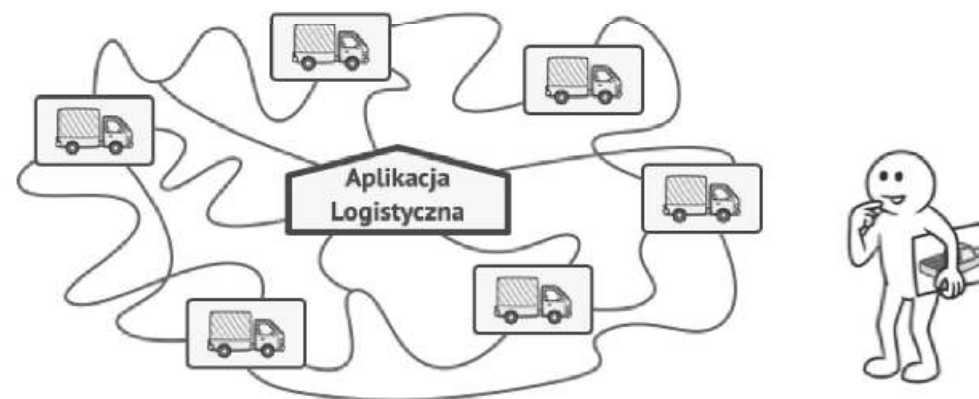
Kreacyjny wzorzec projektowy, który udostępnia interfejs do tworzenia obiektów w ramach klasy bazowej, ale pozwala podklasom zmieniać typ tworzonych obiektów.

Złożoność ★★
Popularność ★★



Motywacja

Wyobraź sobie, że tworzysz aplikacje do zarządzania logistyką. Na razie aplikacja obsługuje tylko transport kołowy, a dokładniej ciężarówki. Aplikacja staje się popularna i pojawia się coraz więcej zgłoszeń od firm spedycyjnych, że przydałaby się również obsługa transportu wodnego.

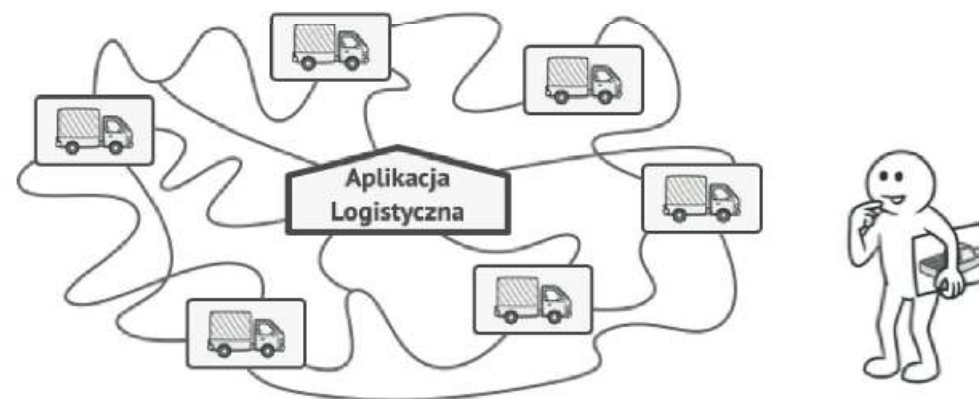


Dodanie nowej klasy do programu nie jest takie proste, jeśli reszta kodu jest już silnie powiązana z istniejącą klasą/klasami.



Rozwiązanie

Wyobraź sobie, że tworzysz aplikacje do zarządzania logistyką. Na razie aplikacja obsługuje tylko transport kołowy, a dokładniej ciężarówki. Aplikacja staje się popularna i pojawia się coraz więcej zgłoszeń od firm spedycyjnych, że przydałaby się również obsługa transportu wodnego.



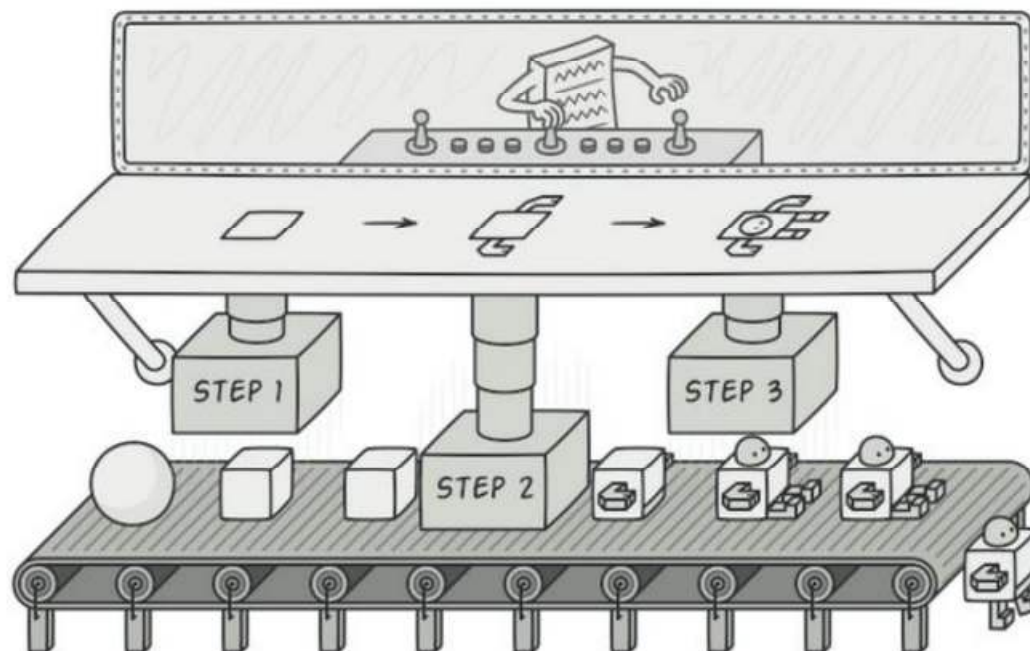
Dodanie nowej klasy do programu nie jest takie proste, jeśli reszta kodu jest już silnie powiązana z istniejącą klasą/klasami.



Fabryka abstrakcyjna (Builder)

Kreacyjny wzorzec projektowy, którego celem jest rozdzielenie sposobu tworzenia obiektu od jego reprezentacji.

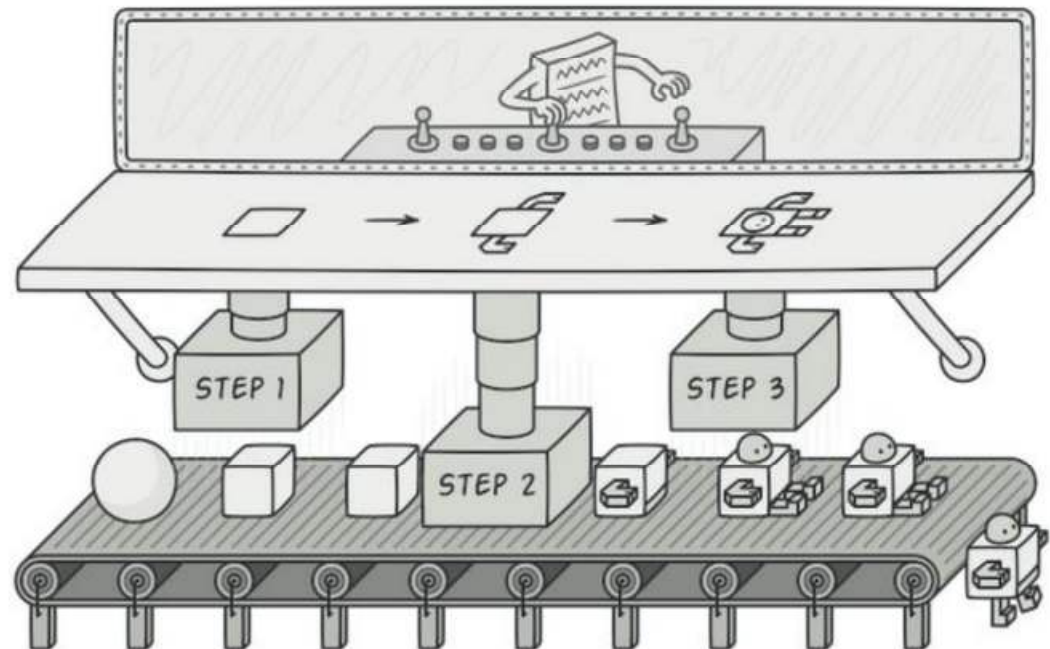
Złożoność ★★☆☆
Popularność ★★★★★



Budowniczy (Builder)

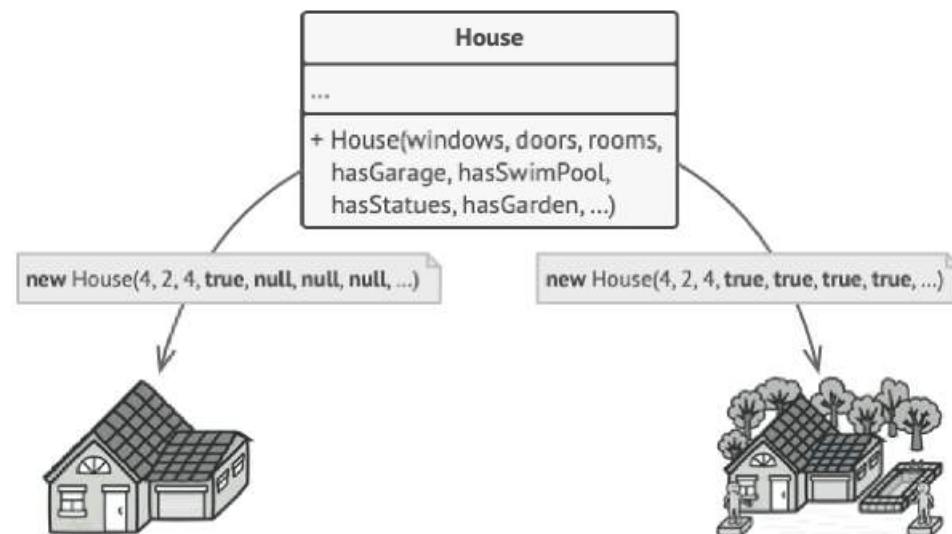
Kreacyjny wzorzec projektowy, którego celem jest rozdzielenie sposobu tworzenia obiektu od jego reprezentacji.

Złożoność ★★☆☆
Popularność ★★★★★



Motywacja

Wyobraź sobie jakiś skomplikowany obiekt (np. dom), którego inicjalizacja jest pracochłonnym, wieloetapowym procesem obejmującym wiele pól i obiektów zagnieżdżonych.

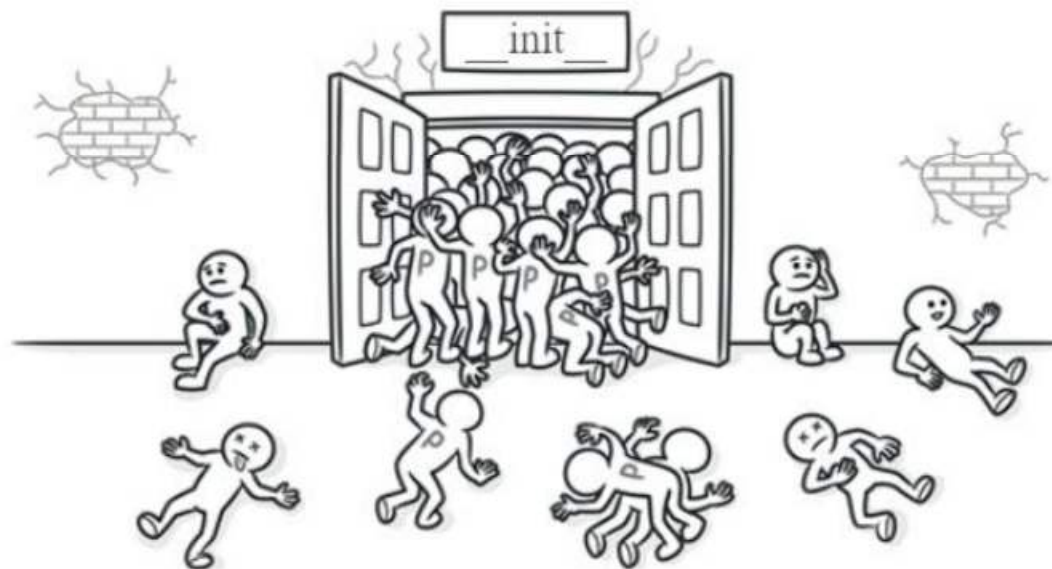


Konstruktor przyjmujący mnóstwo parametrów ma swoją wadę: nie wszystkie parametry będą potrzebne za każdym razem.



Trudności

Taki kod inicjalizacyjny jest często wrzucany do wielgachnego konstruktora, przyjmującego mnóstwo parametrów. Albo jeszcze gorzej, kod taki bywa rozrzucony po całym kodzie klienckim.

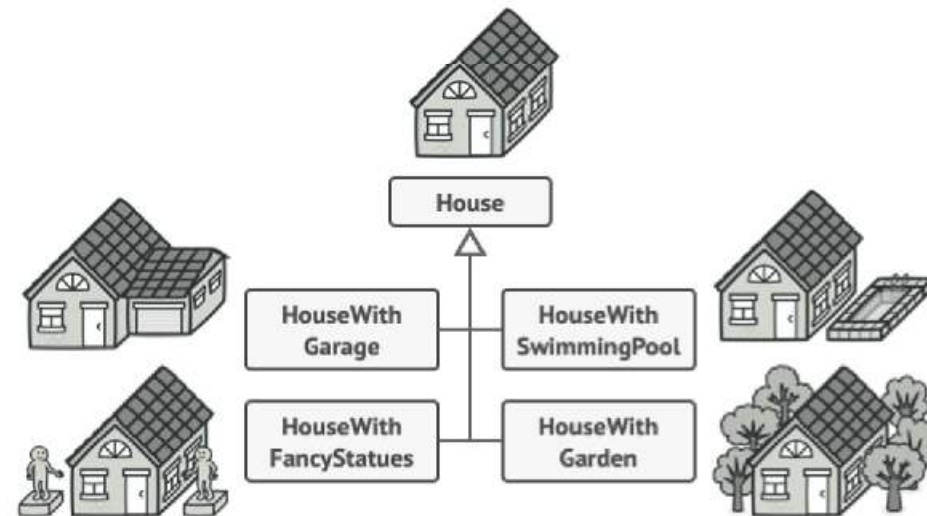


Konstruktor przyjmujący mnóstwo parametrów ma swoją wadę: nie wszystkie parametry będą potrzebne za każdym razem.



Próba uporządkowania

Możemy rozszerzyć klasę bazową Dom i stworzyć system podklas, które spełniałyby każdy możliwy zestaw wymogów. Ale takie podejście doprowadzi do wielkiej liczby podklas. Dodanie kolejnego parametru, jak styl werandy, jeszcze bardziej rozbuduje tę hierarchię.



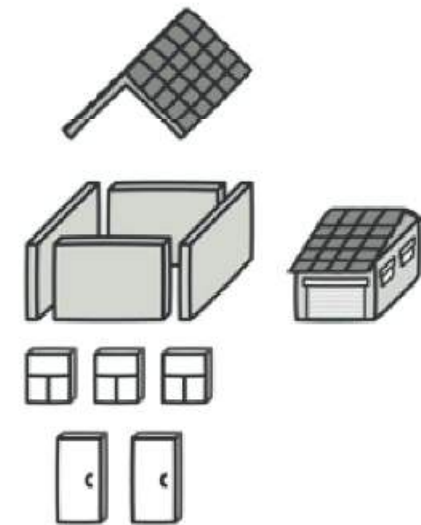
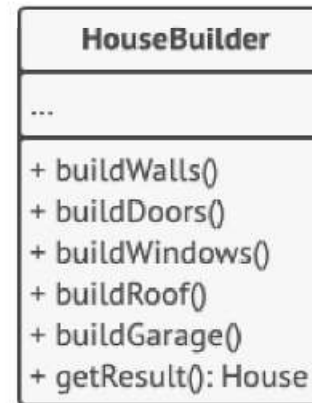
Program może stać się nadmiernie skomplikowany, jeśli każda możliwa konfiguracja oznacza dodanie nowej podklasy.



Rozwiązanie

Wzorec projektowy Budowniczy proponuje wydzielenie kodu konstrukcyjnego obiektu z jego klasy i umieszczenie go w osobnym obiekcie zwany *budowniczym*.

Następnie wydzielony kod konstrukcyjny można podzielić na etapy (budujŚciany, wstawDrzwi, itp) i wywoływać tylko te etapy, które są niezbędne dla określonej konfiguracji obiektu. W ten sposób unikamy dużej liczby podklas.



Wzorec Budowniczy pozwala konstruować złożone obiekty krok po kroku



Interfejs budowniczego

Niektóre etapy konstrukcji mogą wymagać odmiennych implementacji reprezentacji produktu. Na przykład, ściany leśnej chatki mogą być drewniane, ale mury zamku warownego — kamienne.

W takim przypadku, można utworzyć wiele różnych klas budowniczych które implementują te same etapy konstrukcji, ale w różny sposób. Część etapów można wynieść do wspólnego interfejsu.



Różni budowniczowie wykończą to samo zadanie w różny sposób



Kierownik (director)

Poszczególne etapy konstrukcji można wywoływać w odpowiedniej kolejności z poziomu kodu klienckiego. Można też przenieść te wywołania do osobnej klasy, zwanej *kierownikiem*.

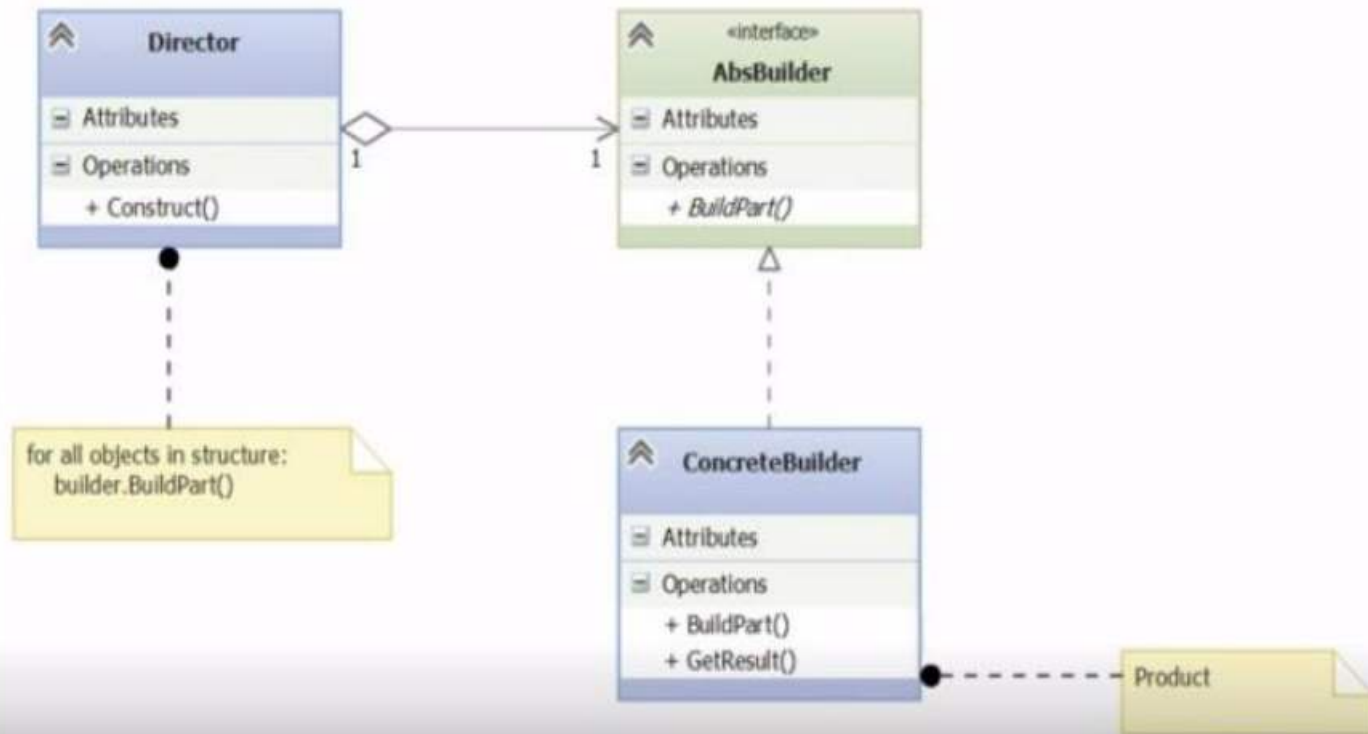
Kierownik określa kolejność etapów jaką musi zachować budowniczy. Wtedy kod kliencki musi tylko skojarzyć budowniczego z kierownikiem, a następnie odebrać wynik pracy od kierownika.



Kierownik wie jakie kroki należy wykonać, aby otrzymać działający produkt.

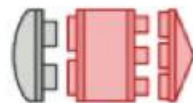


Struktura wzorca Budowniczy (Builder)



Wzorce strukturalne

(Structural Patterns)



Adapter

Adapter



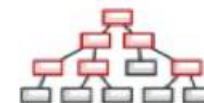
Fasada

Facade



Dekorator

Decorator



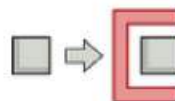
Kompozyt

Composite



Most

Bridge



Proxy



Pyłek

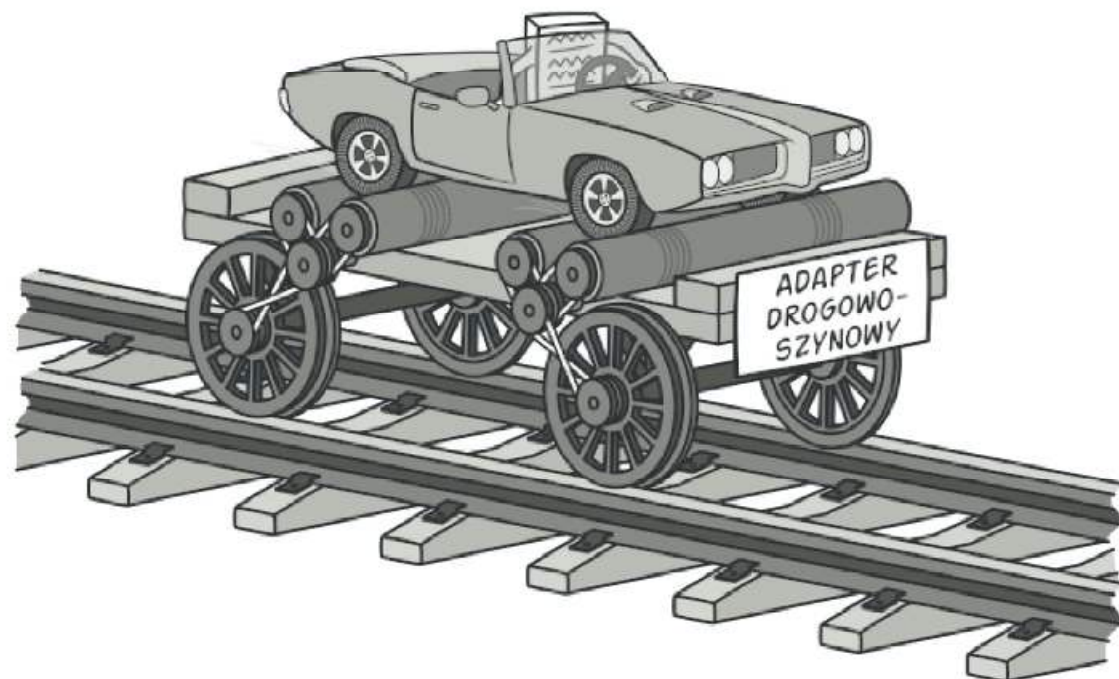
Flyweight

Adapter

Strukturalny wzorzec projektowy pozwalający na współdziałanie ze sobą obiektów o niekompatybilnych interfejsach.

Złożoność ★★☆☆

Popularność ★★★★★



Adapter

Adapters in Real Life



Wall wart

Adapter

Adapters in Real Life



Wall wart



Pipe adapter

Adapter

Adapters in Real Life



Wall wart

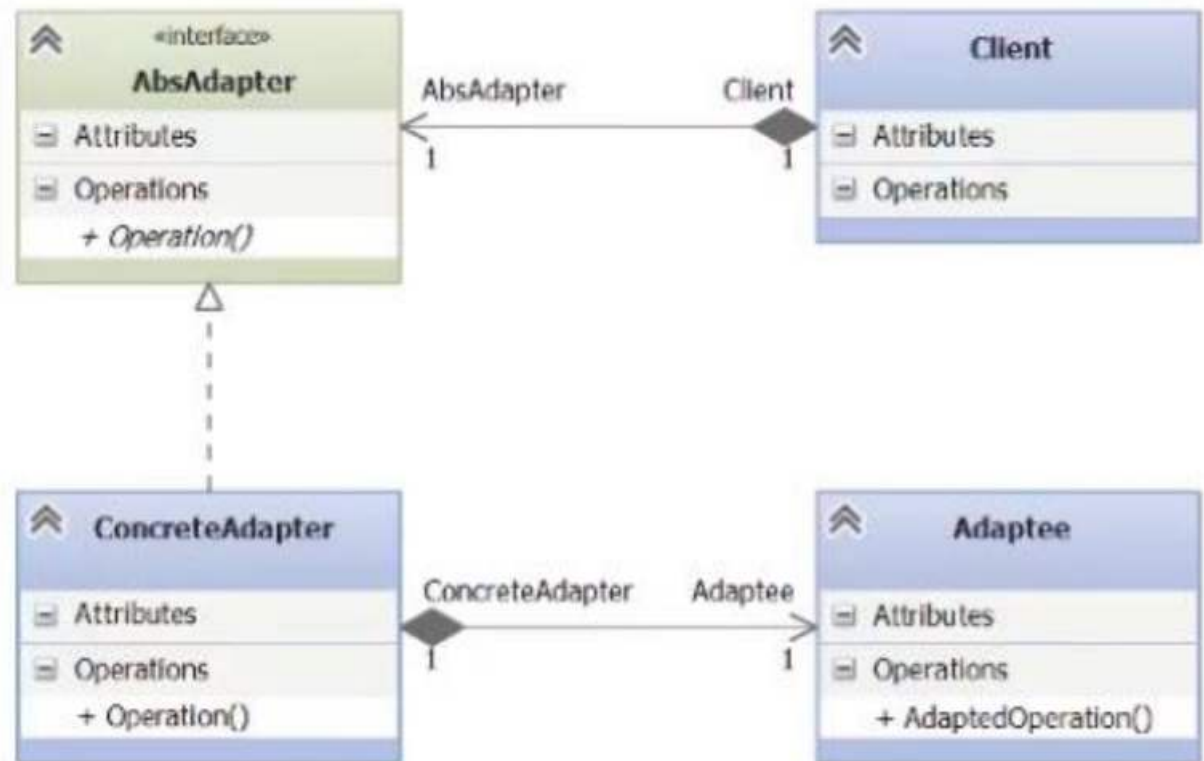


Pipe adapter



Don't try this at home!

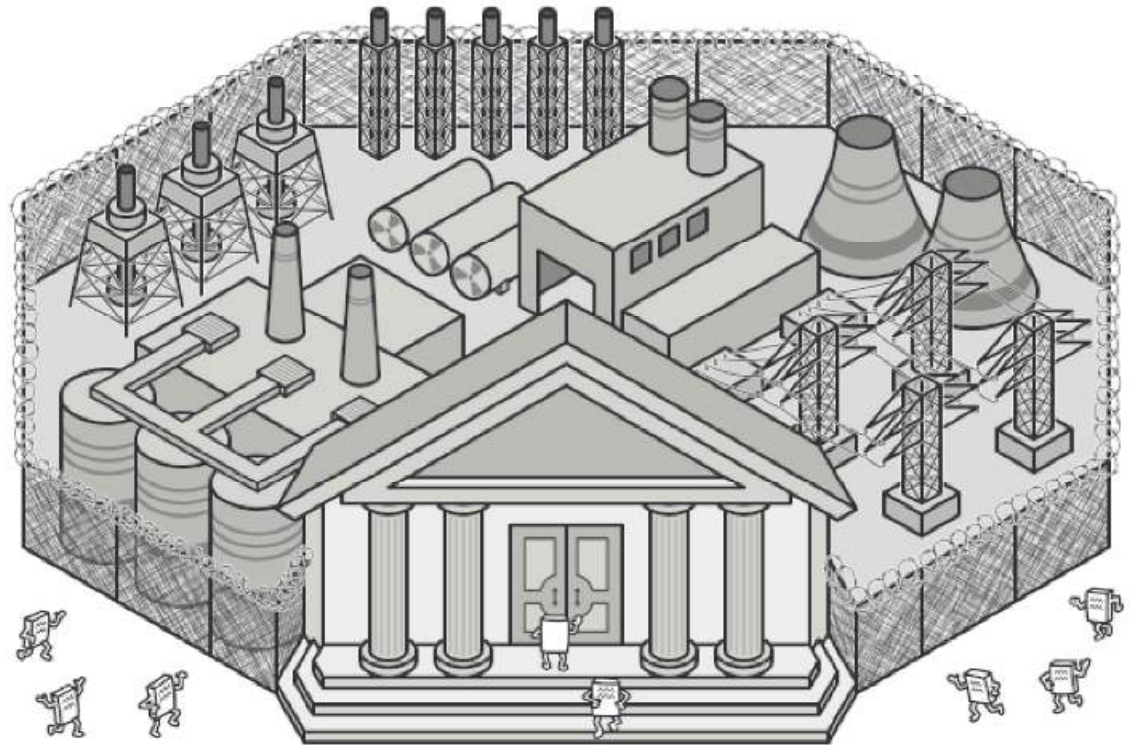
Struktura wzorca adapter (Adapter aka Wrapper)



Fasada (Facade)

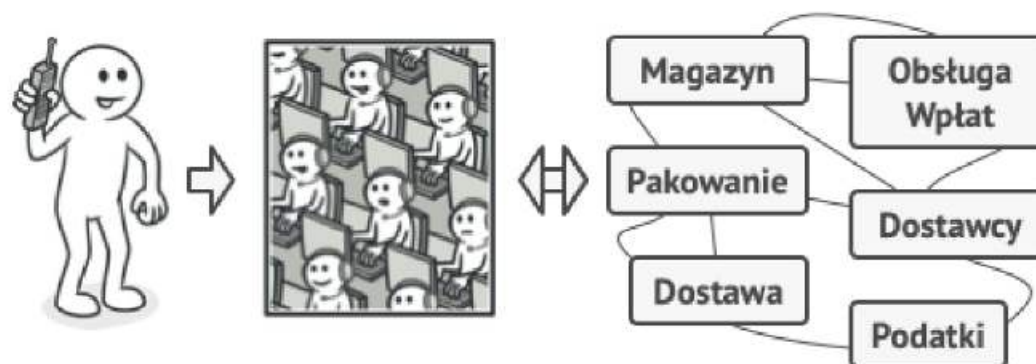
Strukturalny wzorzec projektowy,
który wyposaża bibliotekę,
framework lub inny złożony
zestaw klas w uproszczony interfejs

Złożoność ★★☆☆
Popularność ★★☆☆



Analogia z życia

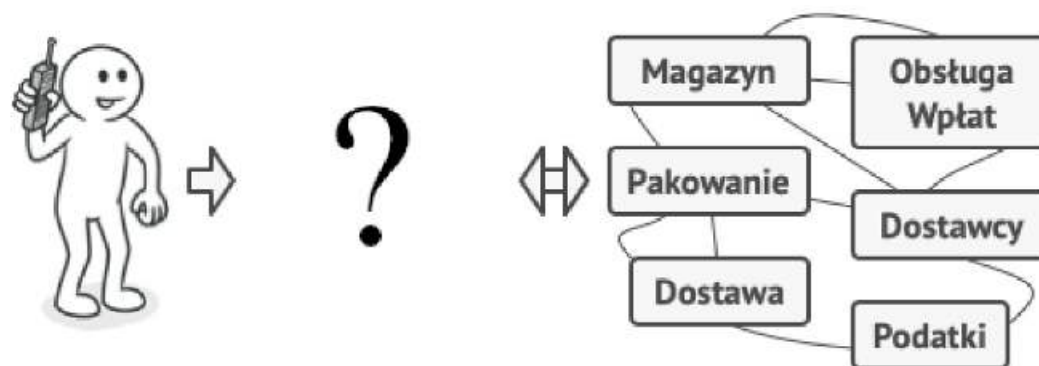
Gdy dzwonisz do sklepu aby złożyć zamówienie, biuro jest twoją fasadą dla wszystkich usług i oddziałów tego sklepu. Pracownik sklepu, czy automat zgłoszeniowy, stanowią prosty interfejs głosowy do systemu zamawiania, płacenia i różnych usług dostawczych.



Składanie zamówienia przez telefon



A jak by to wyglądało bez fasady ?



Składanie zamówienia przez telefon



A jak by to wyglądało bez fasady ?



Składanie zamówienia przez telefon



Motywacja

Wyobraź sobie, że twój kod musi współdziałać z szerokim zestawem obiektów należących do jakiejś skomplikowanej biblioteki lub frameworku. Zazwyczaj należy zainicjalizować wszystkie te obiekty, śledzić zależności, wywoływać metody w odpowiedniej kolejności i tak dalej.

W rezultacie doszłoby do ścisłego sprzęgnięcia logiki biznesowej twoich klas ze szczegółami implementacji klas innych dostawców, co utrudniłoby utrzymanie i zrozumienie kodu.



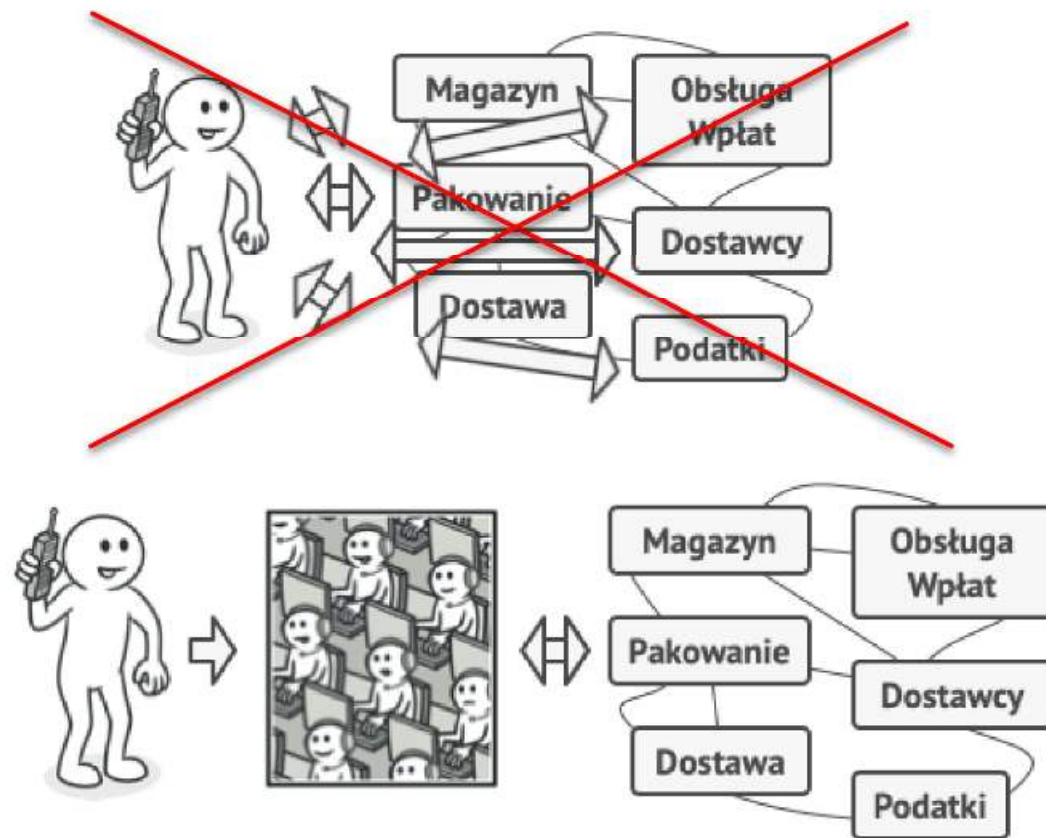
Składanie zamówienia przez telefon



Rozwiązanie

Fasada to klasa stanowiąca prosty interfejs dla złożonego podsystemu, zawierającego mnóstwo ruchomych części. Fasada może dawać ograniczoną funkcjonalność, w porównaniu z korzystaniem z elementów podsystemu bezpośrednio, ale za to eksponuje tylko te możliwości, których klient naprawdę potrzebuje.

Klient korzysta z fasady zamiast wywoływać obiekty systemu bezpośrednio.



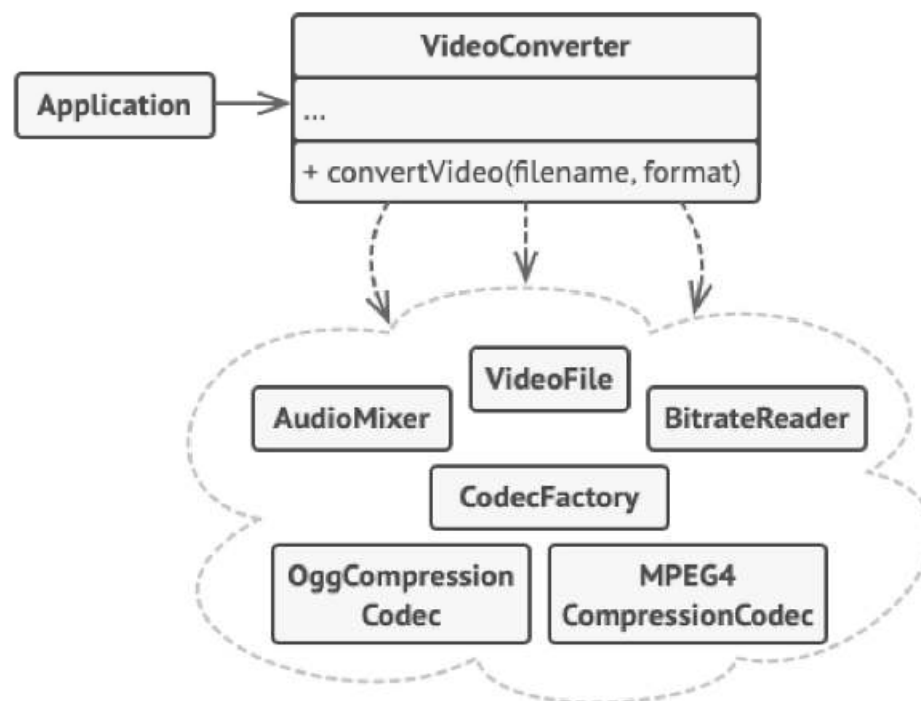
Składanie zamówienia przez telefon



Przykład

W przykładzie wzorzec **Fasada** upraszcza interakcję ze złożonym frameworkiem do konwersji wideo.

Zamiast wiązać bezpośrednio kod z wieloma klasami składowymi frameworku, tworzymy klasę fasady która hermetyzuje funkcjonalność i ukrywa ją przed resztą kodu. Ta struktura pozwala też zminimalizować wysiłek związany z aktualizacją frameworku do nowszej wersji lub wręcz wymiany na inny. Jedyna rzecz, jaką trzeba będzie wówczas zmienić, to implementacja metod fasady.



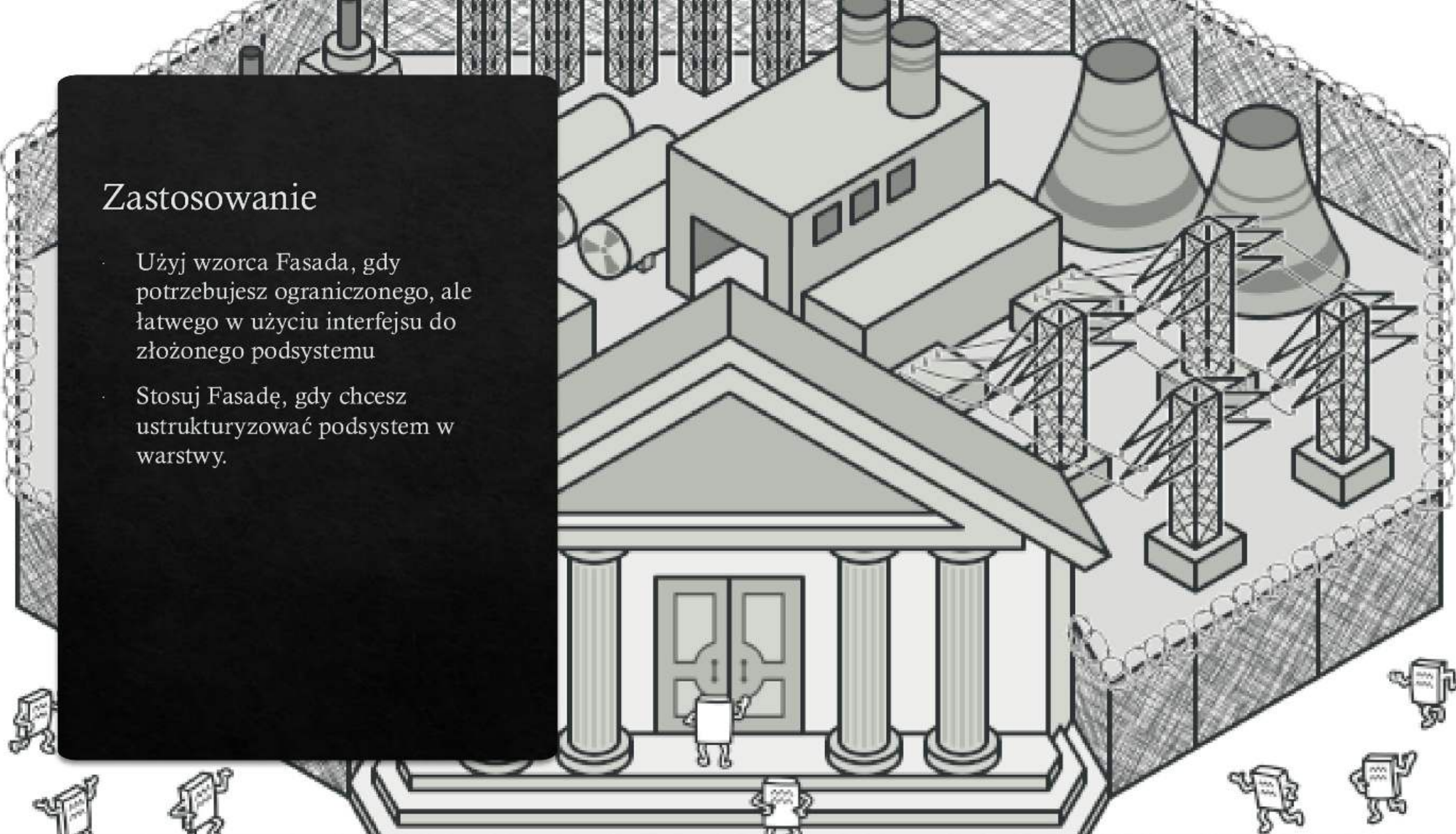
Przykład izolowania wielu zależności w pojedynczej klasie fasady



Zastosowanie

Użyj wzorca Fasada, gdy potrzebujesz ograniczonego, ale łatwego w użyciu interfejsu do złożonego podsystemu

Stosuj Fasadę, gdy chcesz ustrukturyzować podsystem w warstwy.



Kiedy użyć

- Użyj wzorca Fasada, gdy potrzebujesz ograniczonego, ale łatwego w użyciu interfejsu do złożonego podsystemu
- Stosuj Fasadę, gdy chcesz ustrukturyzować podsystem w warstwy.

Jak nie używać

- Uważaj, żeby Fasada nie stała się boskim obiektem, sprzężonym ze wszystkimi klasami aplikacji.



Wzorce behawioralne

(Behavioral Patterns)



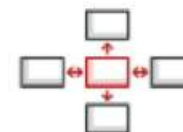
**Łańcuch
zobowiązań**
Chain of
Responsibility



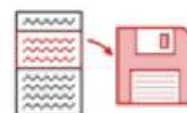
Polecenie
Command



Iterator
Iterator



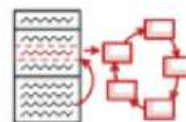
Mediator
Mediator



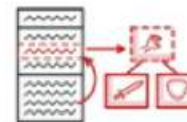
Pamiętka
Memento



Obserwator
Observer



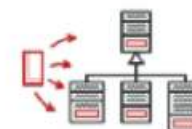
Stan
State



Strategia
Strategy



**Metoda
szablonowa**
Template Method

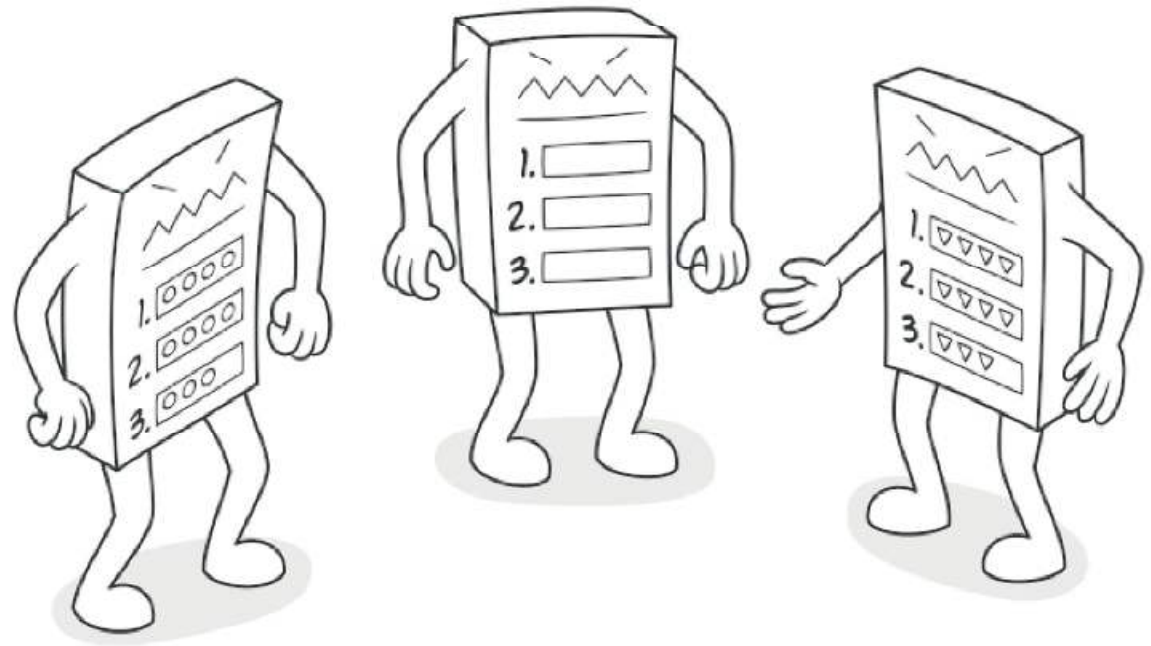


Odwiedzający
Visitor

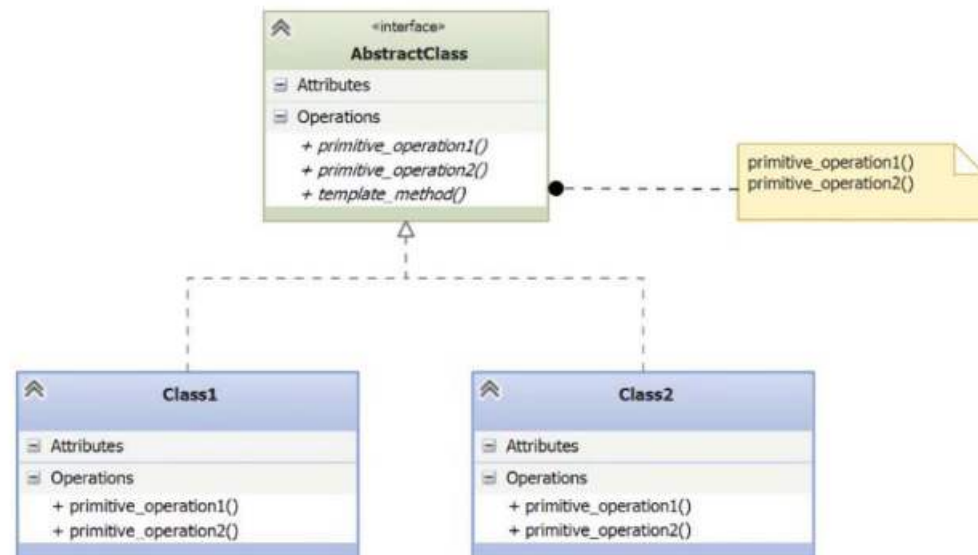
Metoda szablonowa (Template)

Czynnościowy wzorzec projektowy definiujący szkielet algorytmu w klasie bazowej, ale pozwalający podklasom nadpisać pewne etapy tego algorytmu bez konieczności zmiany jego struktury.

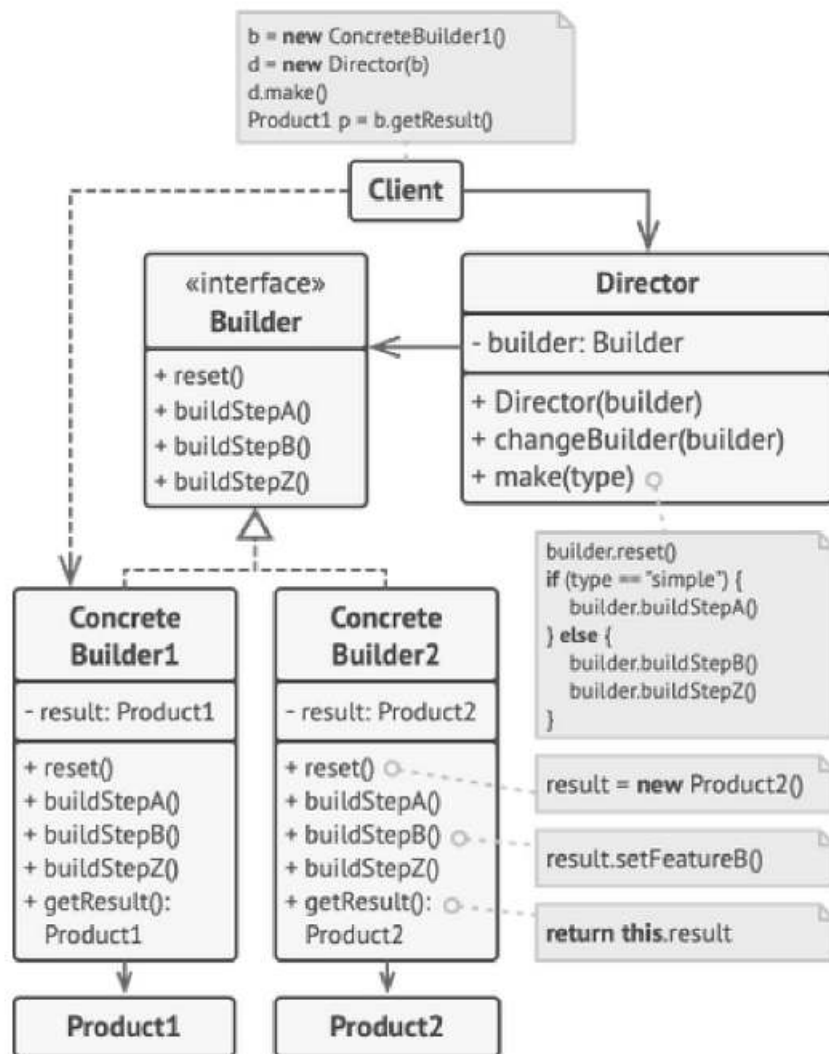
Złożoność ★★
Popularność ★★



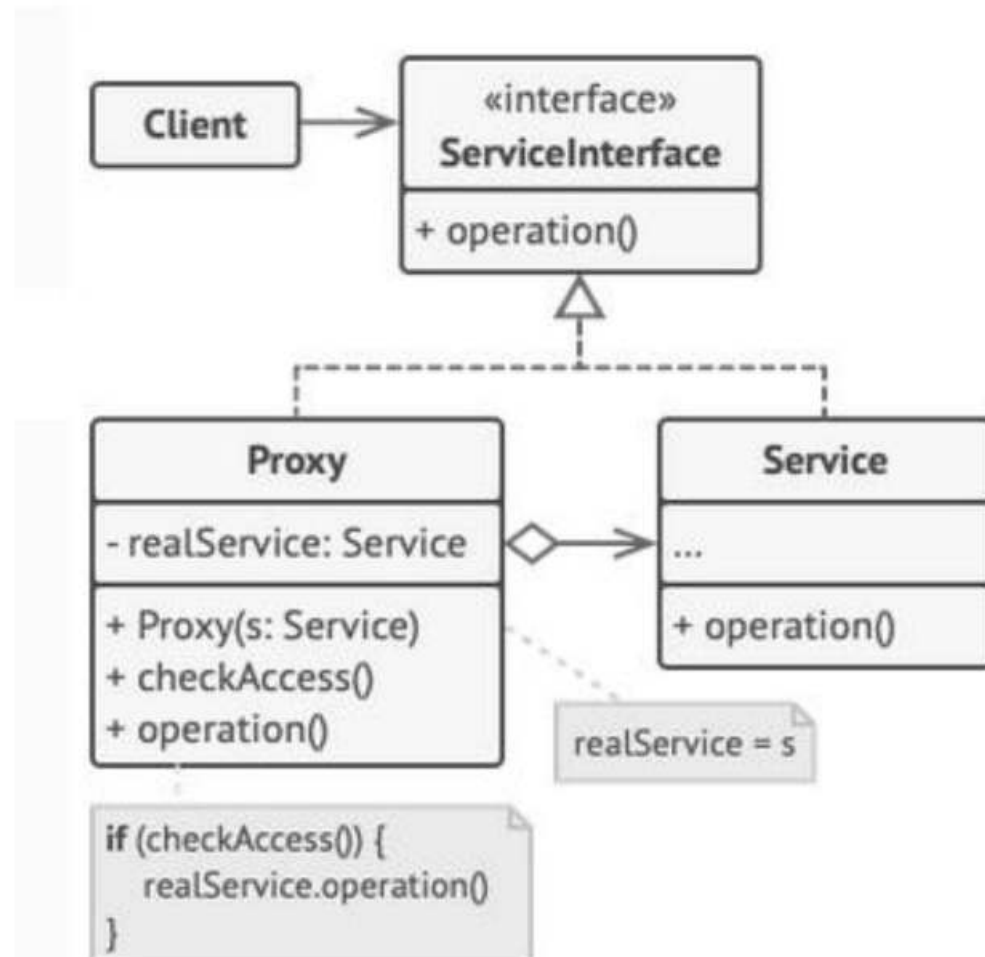
Struktura wzorca metoda szablonowa (Template)



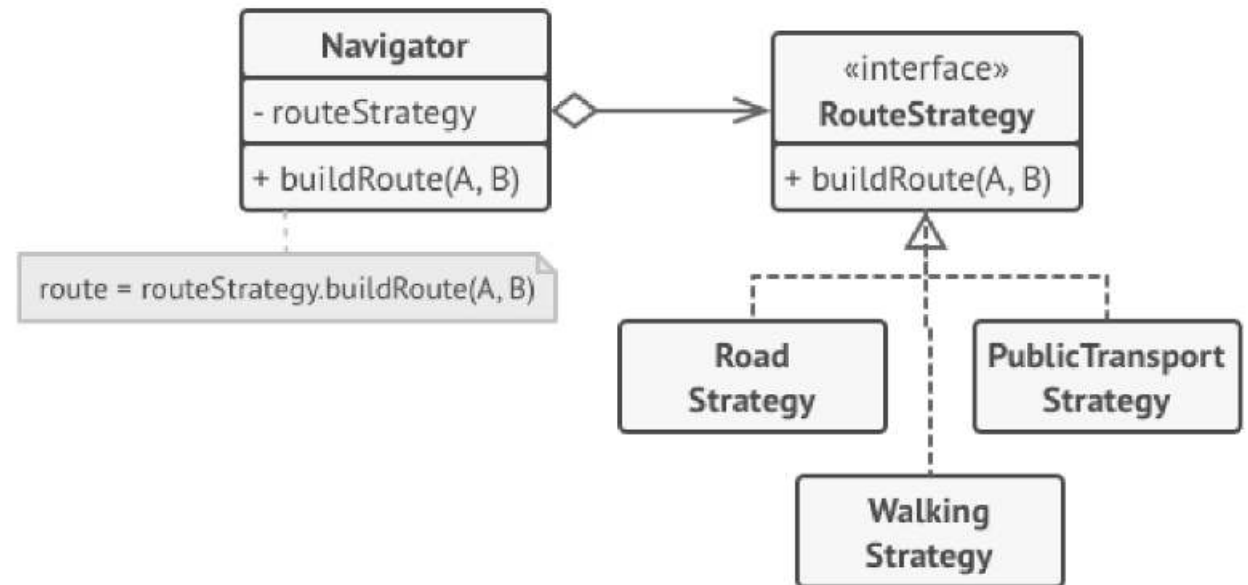
Budowniczy



Proxy

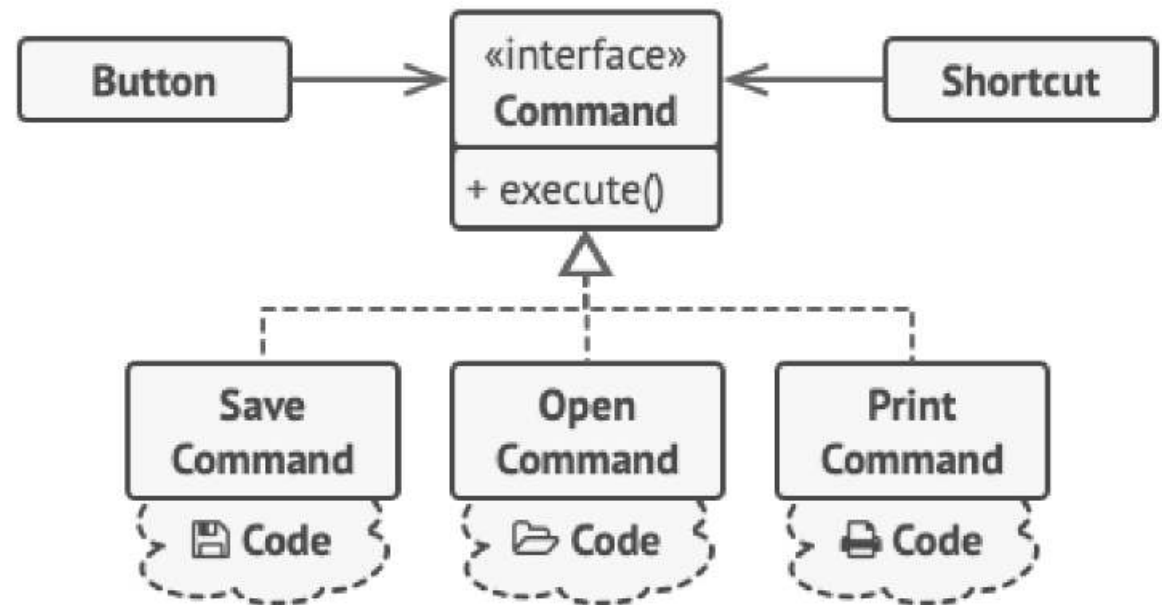


Strategia



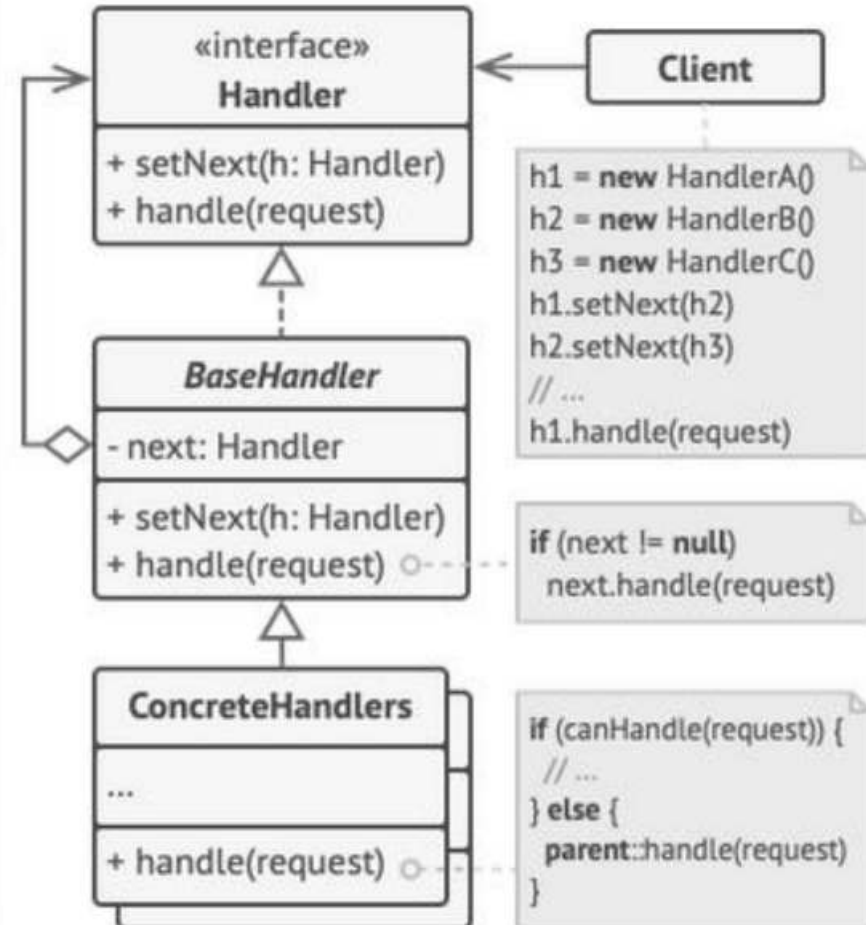
Route planning strategies.

Komenda

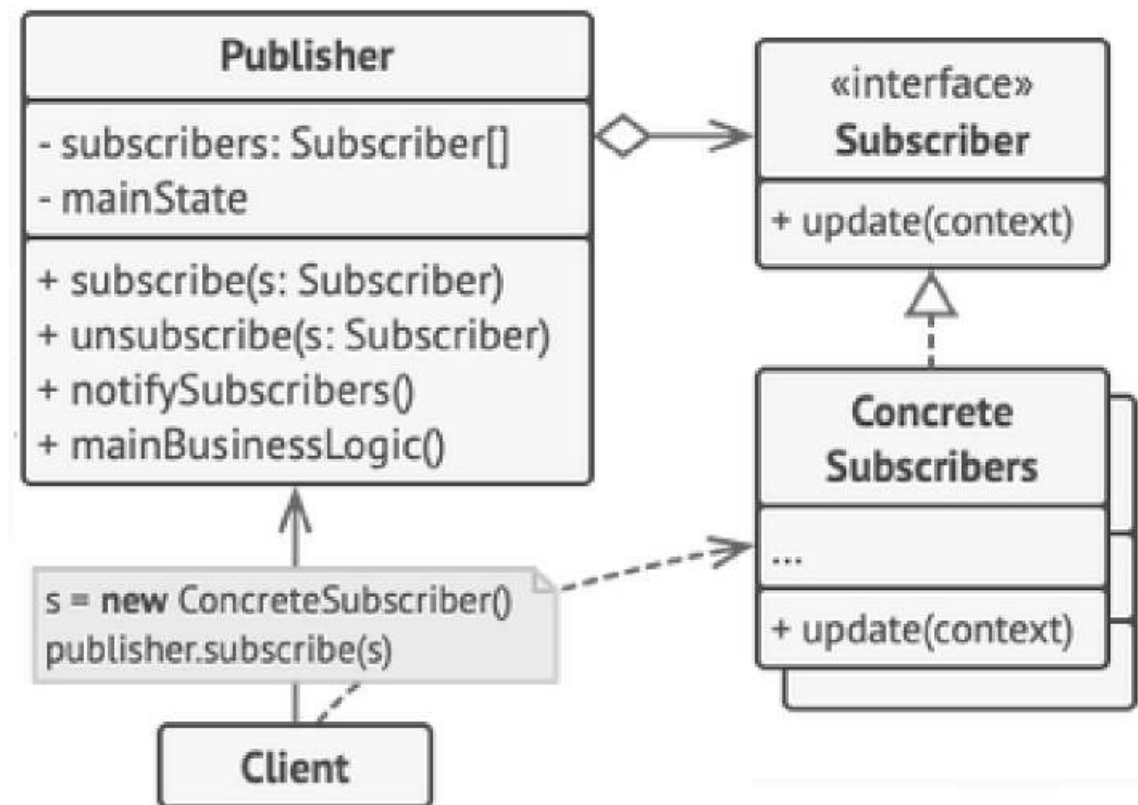


The GUI objects delegate the work to commands.

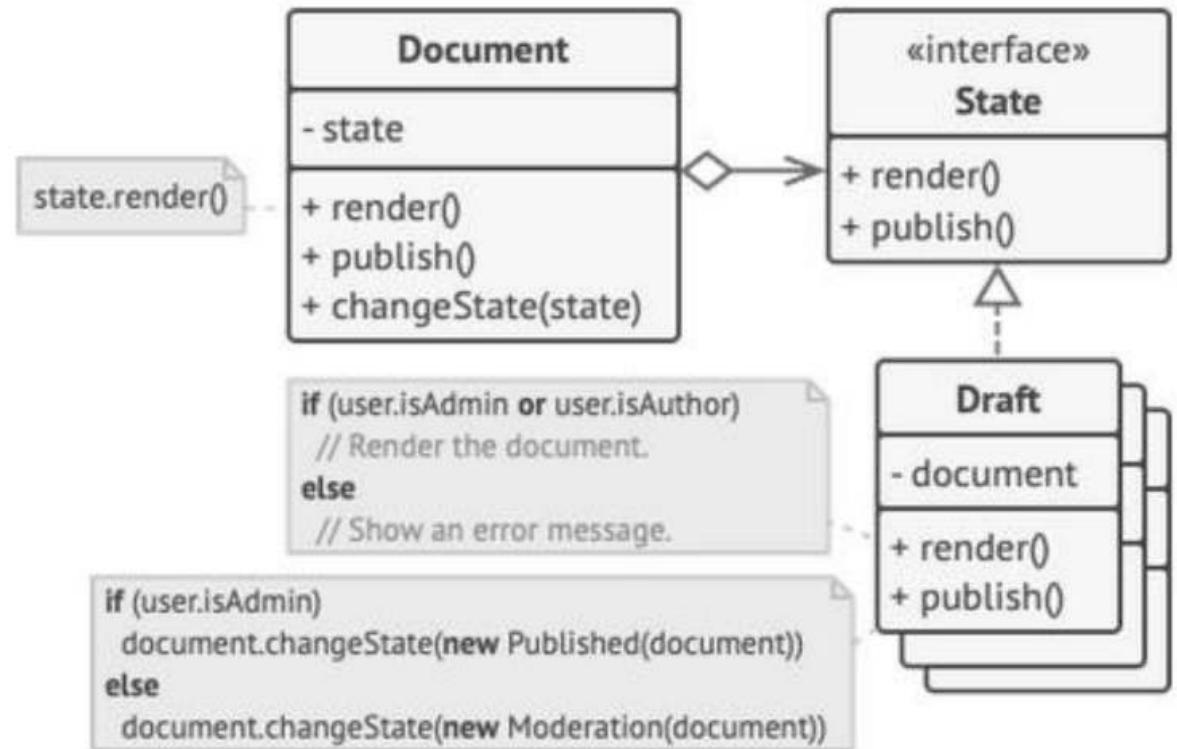
Łańcuch odpowiedzialności



Observer



Stan



Document delegates the work to a state object.

Zapachy kodu

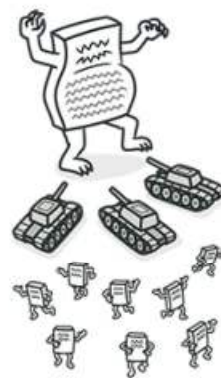
Zapachy kodu

Cechy kodu mówiące o złym sposobie implementacji i będące sygnałem do refaktoryzacji.

Zapachy kodu

Najpopularniejsza obecnie klasyfikacja, wprowadzona przez Mika Mantyla w artykule: A Taxonomy for "Bad Code Smells" (2006 r.) wyróżnia pięć kategorii zapachów kodu.

Klasyfikacja Mantyla



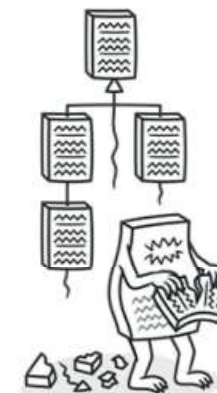
Bloaters



Dispensables



Change Preventers



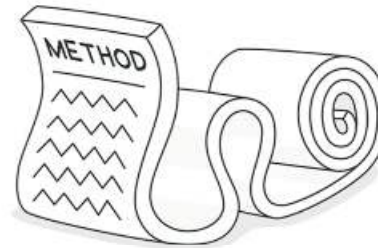
Object-Orientation Abuser



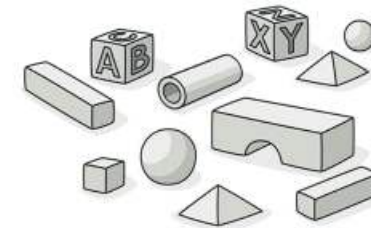
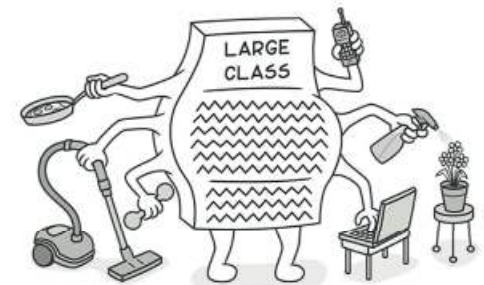
Couplers

Bloaters	Object-Orientation Abusers	Change Preventers	Dispensables	Couplers
• Long Method • Large Class • Primitive Obsession • Long Parameter List • Data Clumps	• Alternative Classes with Different Interface • Refused Bequest • Switch Statements • Temporary Field	• Divergent Change • Parallel Inheritance Hierarchies • Shotgun Surgery	• Comments • Duplicate Code • Data Class • Dead Code • Lazy Class • Speculative Generality	• Feature Envy • Inappropriate Intimacy • Incomplete Library Class • Message Chains • Middle Man

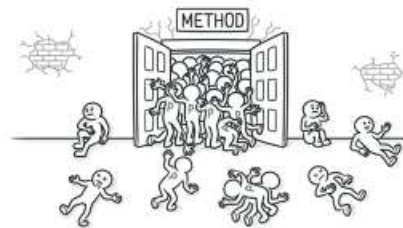
Bloaters



Long Method



Primitive Obsession

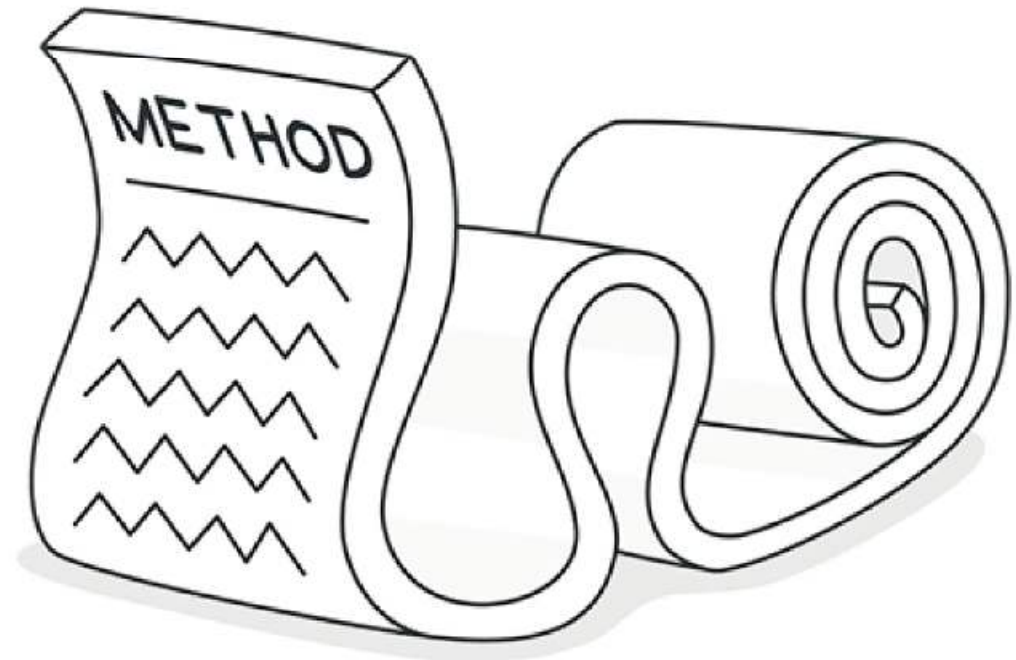


Long Parameter List

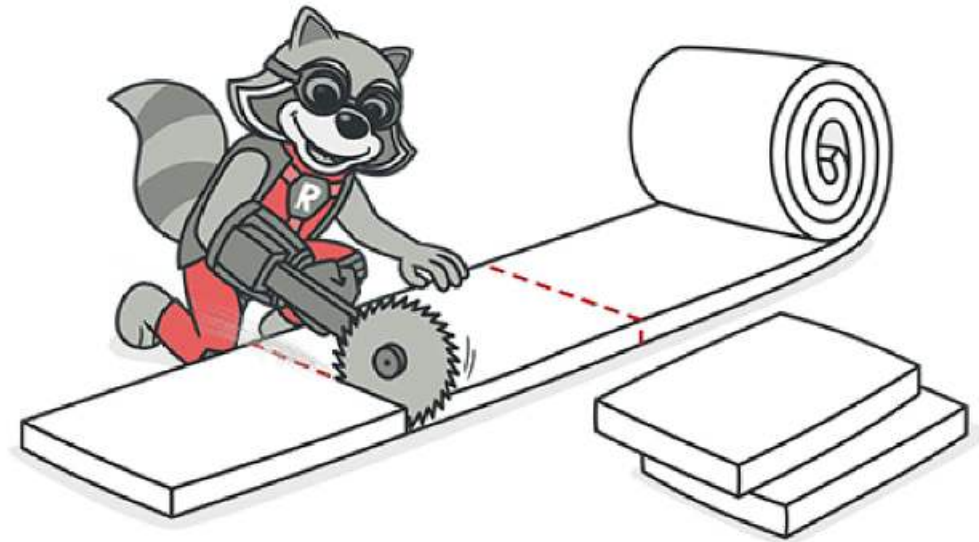


Data Clumps

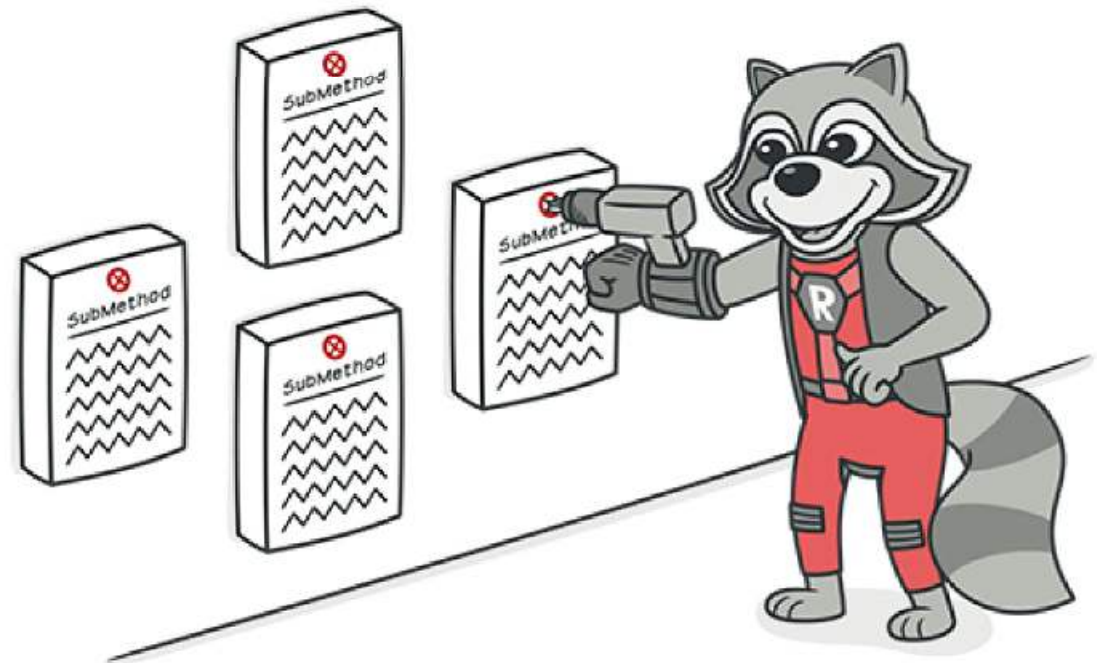
Long method



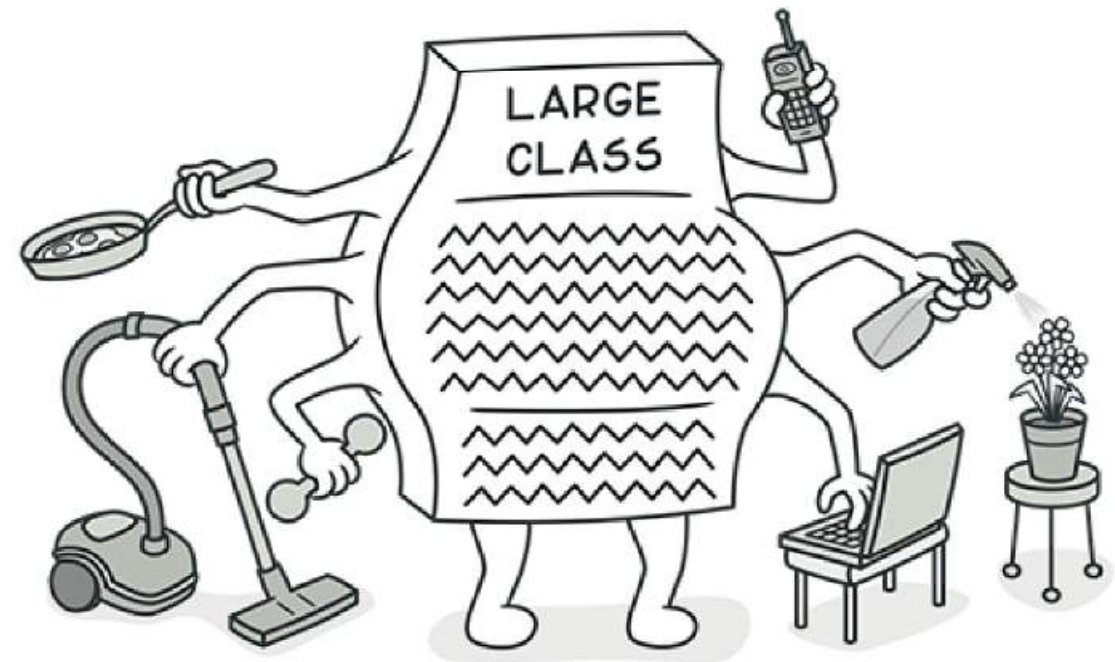
Long method



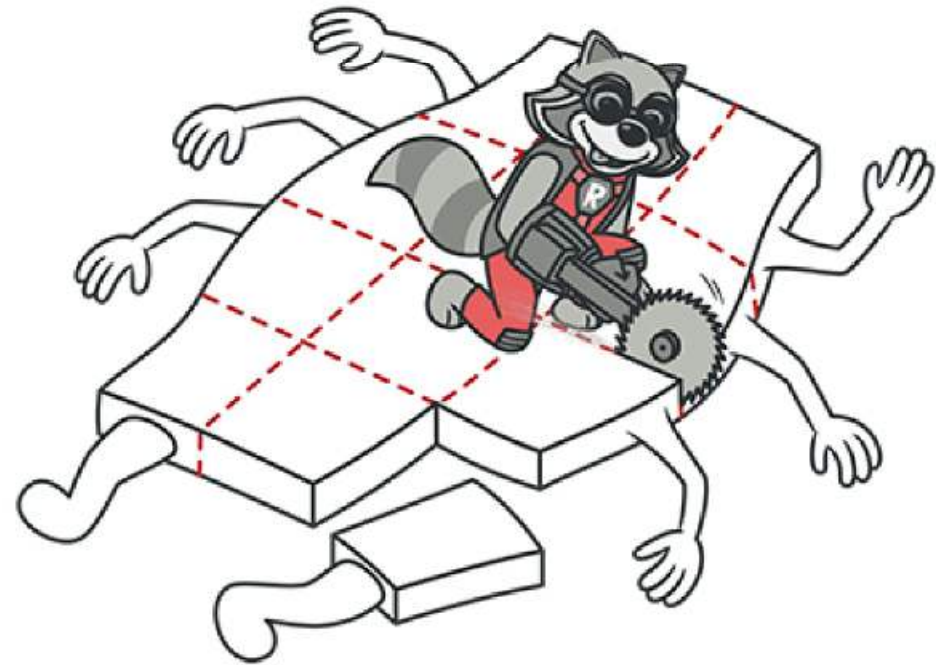
Long method



Large class



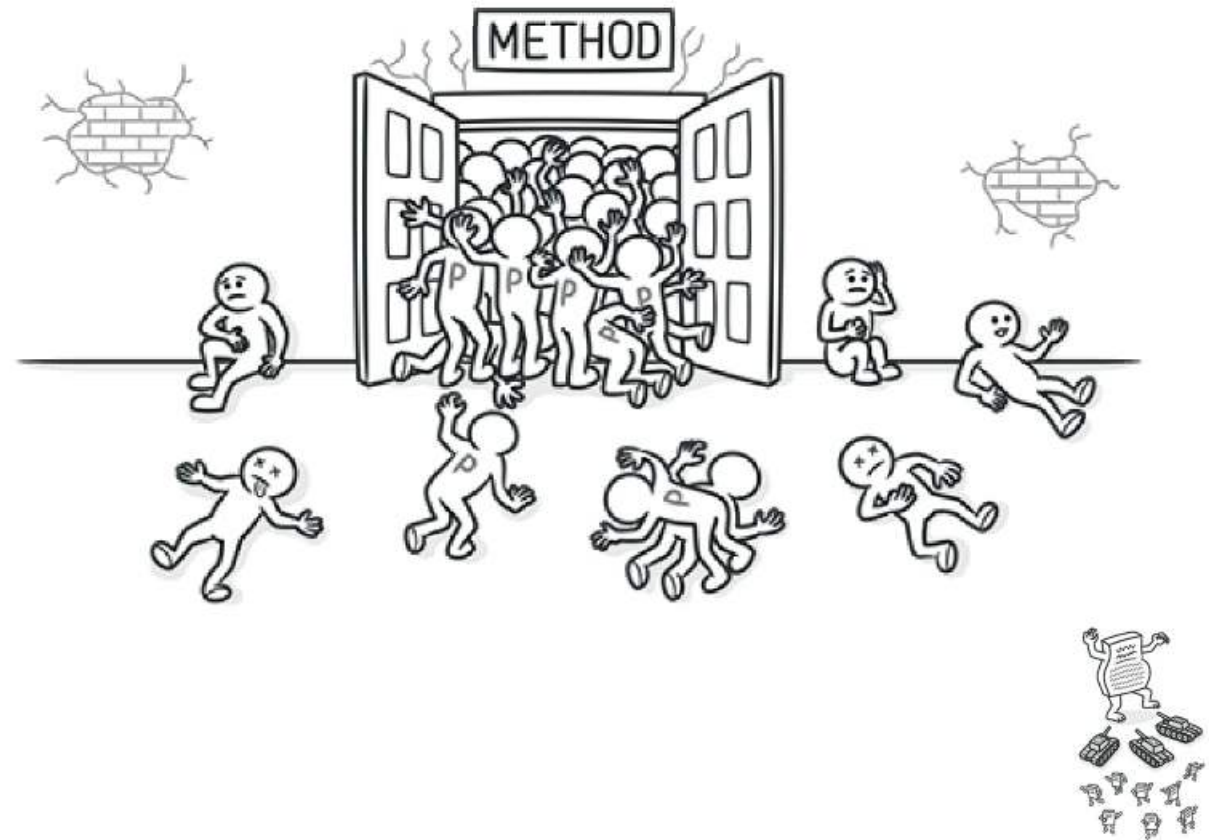
Large class



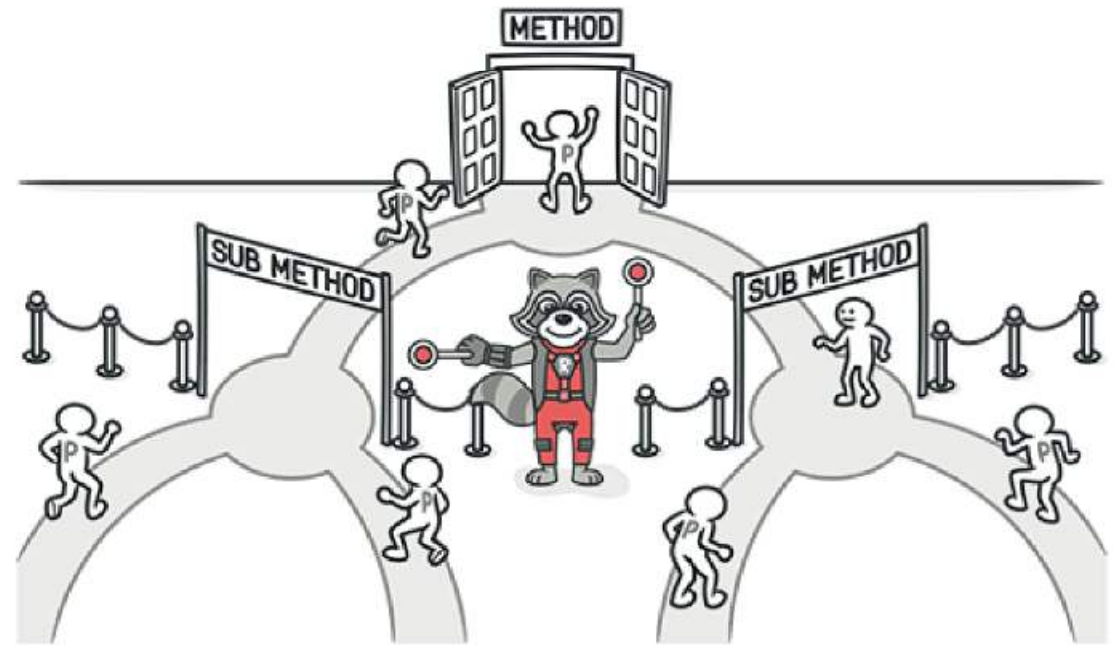
Large class



Long parameter list



Long parameter list



Dobre praktyki



PEP – Python Enhancement Proposals

PEP – Python
Enhancement Proposals

Co to PEP ?

PEP – Python Enhancement Proposals

Co to PEP ?

Zbiór oficjalnych, numerowanych dokumentów informacyjnych społeczność Pythona o wprowadzanych zmianach, propozycjach zmian, przyjętych konwencjach i standardach.

PEP 1

PEP – Python Enhancement Proposals

◇ <https://www.python.org/dev/peps>

PEP – Python Enhancement Proposals

- ◇ <https://www.python.org/dev/peps>
- ◇ PEP 8 – Style Guide for Python Code

PEP – Python Enhancement Proposals

- ◇ <https://www.python.org/dev/peps> - pełna lista
- ◇ PEP 8 – Style Guide for Python Code
- ◇ PEP 257 – Docstring Conventions

PEP – Python Enhancement Proposals

- ◇ <https://www.python.org/dev/peps>
- ◇ PEP 8 – Style Guide for Python Code
- ◇ PEP 257 – Docstring Conventions
- ◇ PEP 318 – Decorators for Functions nad Methods

PEP 8

A thin, vertical white line is positioned to the right of the text 'PEP 8', extending from approximately the middle of the text's vertical range down to the bottom of the frame.

PEP 8

◇ Style Guide for Python Code:

PEP 8

- ◇ Style Guide for Python Code:
 - ◇ wcięcia za pomocą 4 spacji

PEP 8

- ◇ Style Guide for Python Code:
 - ◇ wcięcia za pomocą 4 spacji
 - ◇ maksymalna długość linii - 79

PEP 8

- ◇ Style Guide for Python Code:
 - ◇ wcięcia za pomocą 4 spacji
 - ◇ maksymalna długość linii - 79
 - ◇ odległości w kodzie (białe linie i spacje)

PEP 8

- ◇ Style Guide for Python Code:
 - ◇ wcięcia za pomocą 4 spacji
 - ◇ maksymalna długość linii - 79
 - ◇ odległości w kodzie (białe linie i spacje)
 - ◇ komentarze

PEP 8

- ◇ Style Guide for Python Code:
 - ◇ wcięcia za pomocą 4 spacji
 - ◇ maksymalna długość linii - 79
 - ◇ odległości w kodzie (białe linie i spacje)
 - ◇ komentarze
 - ◇ nazewnictwo zmiennych i funkcji – snake_case

PEP 8

- ◇ Style Guide for Python Code:
 - ◇ wcięcia za pomocą 4 spacji
 - ◇ maksymalna długość linii - 79
 - ◇ odległości w kodzie (białe linie i spacje)
 - ◇ komentarze
 - ◇ nazewnictwo zmiennych i funkcji – snake_case
 - ◇ nazewnictwo klas – CamelCase

Google Python Style Guide

